

Curso: Bases de Datos I

Ingeniería en Sistemas y Ciencias de la Computación

Clase 1: Introducción a las Bases de Datos

Catedrático: Claudia Alegre

17 de enero de 2026

Dinámica de Introducción

Conocer al equipo

- Catedrático se presenta
Breve hoja de vida
- Alumnos se presentan
Nombre,
Estudiante/trabajador
Expectativas del curso

Tema

- Introducción a las bases de datos

Indicadores de Logro

- Los estudiantes serán capaces de definir qué es una base de datos y para qué se utiliza.
- Los estudiantes serán capaces de nombrar y describir las bases de datos más comunes.

Actividades

- Explicación de la teoría.
- Diferencia entre una base de datos y un sistema de archivos.
- Componentes básicos de un sistema de gestión de bases de datos (DBMS).
- Validación de conocimientos

Teoría: ¿Qué es una Base de Datos?

Una base de datos es un conjunto de datos relacionados que están almacenados en un sistema estructurado y que pueden ser gestionados de manera eficiente.

Considera la capacidad de almacenar, recuperar, actualizar y eliminar información de forma organizada.

Las bases de datos son fundamentales para cualquier organización que maneje grandes cantidades de datos, ya que permiten estructurar la información de manera que sea fácil de acceder, compartir y analizar.

Teoría: DBMS

Un sistema de administración de datos (Database Management System) es una colección de programas que permite a los usuarios crear y mantener una base de datos.

El DBMS es un sistema de software de propósito general que facilita los procesos de definición, construcción, manipulación y compartición de bases de datos entre varios usuarios y aplicaciones.

Teoría: Metadatos

Definición o información descriptiva de una base de datos también se almacena en forma de catálogo o diccionario de la base de datos.

Definir una base de datos implica especificar:

- Tipos de datos
- Estructuras
- Restricciones de los datos

Teoría: SQL

- Structured Query Language (SQL): Lenguaje de Consulta Estructurado para bases de datos relacionales.
- Estándar ANSI (Instituto Nacional Estadounidense de Estándares) en 1986, y de ISO (Organización Internacional de Normalización) 1987.

Teoría: SQL - Sublenguajes

- **Lenguaje de Consulta de Datos (DQL)** - El Lenguaje de Consulta de Datos es el sublenguaje responsable de leer, o consultar, datos de una base de datos.
SELECT
- **Lenguaje de Manipulación de Datos (DML)** - El Lenguaje de Manipulación de Datos es el sublenguaje responsable de añadir, editar o borrar datos de una base de datos.
INSERT, UPDATE, y DELETE
- **Lenguaje de Definición de Datos (DDL)** - El Lenguaje de Definición de Datos es el sublenguaje responsable de definir la forma en que se estructuran los datos en una base de datos.
CREATE TABLE, ALTER TABLE, y DROP TABLE
- **Lenguaje de Control de Datos (DCL)** - El Lenguaje de Control de Datos es el sublenguaje responsable de las tareas administrativas de control de la propia base de datos, especialmente la concesión y revocación de permisos de base de datos para los usuarios.
GRANT, REVOKE, y DENY

Actividad en Grupos

Base de datos y sistema de archivos.

Elaborar tabla de diferencias y similitudes.

Antes de Concluir la Sesión

- ¿Qué aprendí hoy?
- ¿He visto aplicaciones en la vida real de los puntos vistos hoy?
- ¿Cuál fue el punto más importante que vimos en la clase de hoy?

Tarea para el Viernes Próximo

DBA, responsabilidades y perfil

Curso: Bases de Datos I

Ingeniería en Sistemas y Ciencias de la Computación

Clase 2: Modelo Relacional

Catedrático: Claudia Alegre

24 de enero de 2026

Dinámica de Introducción

Preferencias:

Por alumno:

Hacer una lista de Restaurantes

Hacer una lista de comidas favoritas

Ahora escribir por restaurante que comidas de las listadas hay en cada uno

Tema

Modelos de bases de datos:

- Modelo relacional
- modelo no relacional
- otros modelos

Indicadores de Logro

- Los estudiantes serán capaces de comparar y contrastar los modelos relacional y no relacional, así como otros modelos de bases de datos, e identificar las ventajas y desventajas de cada uno en diferentes contextos.
- Los estudiantes serán capaces de justificar un esquema de almacenamiento apropiado para un caso de uso específico, teniendo en cuenta las necesidades de rendimiento, escalabilidad y distribución.

Actividades

- Explicación de la teoría.
- Ejemplos
- Validación de conocimientos

Teoría: Tipos de bases de datos

Cada tipo de base de datos tiene estructuras propias, a eso se le llama modelo de datos. Algunos de estos tipos de bases de datos:

Modelo plano

Modelo jerárquico

Modelo de red

Modelo relacional

Teoría: Modelo Plano

En este modelo de base de datos hay dos dimensiones de conjunto de datos. Cada fila representa un registro y cada columna un atributo

Ejemplos:

- Una lista de valores.
- Archivo CSV

Teoría: Modelo de base de datos jerárquica

Similar a la estructura de un árbol, cada enlace es anidado con el fin de conservar datos organizados en un orden particular. Cada nodo puede tener múltiples hijos, pero solo un padre.

Cuando un registro tiene relación con dos o más registros, debe almacenarse varias veces, puesto que un hijo no puede tener varios padres

.....

Ejemplos:

- Carpetas organizadas en MS Windows

Teoría: Modelo de base de datos de Red

Formada por una serie de registros, los cuáles están enlazados entre sí creando una red.

Siempre hay un vínculo entre un padre y un hijo, se mantiene la integridad de los datos.

La principal diferencia entre red-jerárquica, es que las bases jerárquicas solo permite un único nodo padre, mientras que en las bases de red es posible crear varios nodos padre.

La base de datos debe dedicar menos tiempo a mantener registros debido a la eliminación de tablas redundantes, lo que mejora la eficiencia y el rendimiento general.

Ejemplos:

- Sistemas bancarios
- Telecomunicaciones

Teoría: Modelo Relacional

Es un método para estructurar datos utilizando relaciones, mediante estructuras en forma de cuadrícula, que consisten en columnas y filas. Es el principio conceptual de las bases de datos relacionales. Fue propuesto por Edgar F. Codd en 1969.

Se define como el modelo de base de datos que permite agrupar sus elementos de datos en una o más tablas independientes, que pueden relacionarse entre sí utilizando campos comunes a cada tabla relacionada.

Teoría: Características del modelo relacional

- **Composición.** Todos los datos se representan conceptualmente como una disposición ordenada de datos en filas y columnas, llamada relación o tabla.
- **Encabezado y cuerpo.** Cada tabla debe tener un encabezado y un cuerpo. El encabezado es simplemente la lista de columnas. El cuerpo es el conjunto de datos que llena la tabla, organizado en filas.
- **Valores.** Todos los valores son escalares. Es decir, en cualquier posición dada de fila/columna en la tabla, hay solo un valor único.
- **Reglas.** El modelo relacional de base de datos debe elaborarse basado en unas reglas, llamadas de integridad, que garantizan la identificación de las filas y columnas.

Teoría: Elementos del modelo relacional

- **Tupla.** Cada fila de datos es una tupla, conocida también como registro. Cada fila es una n -tupla, pero la “ n -” generalmente se descarta.
- **Columna.** Cada columna de una tupla se llama atributo o campo. La columna representa el conjunto de valores que puede tener un atributo específico.
- **Clave.** Cada fila tiene una o más columnas denominada clave de la tabla.

Teoría: Conceptos del modelo relacional

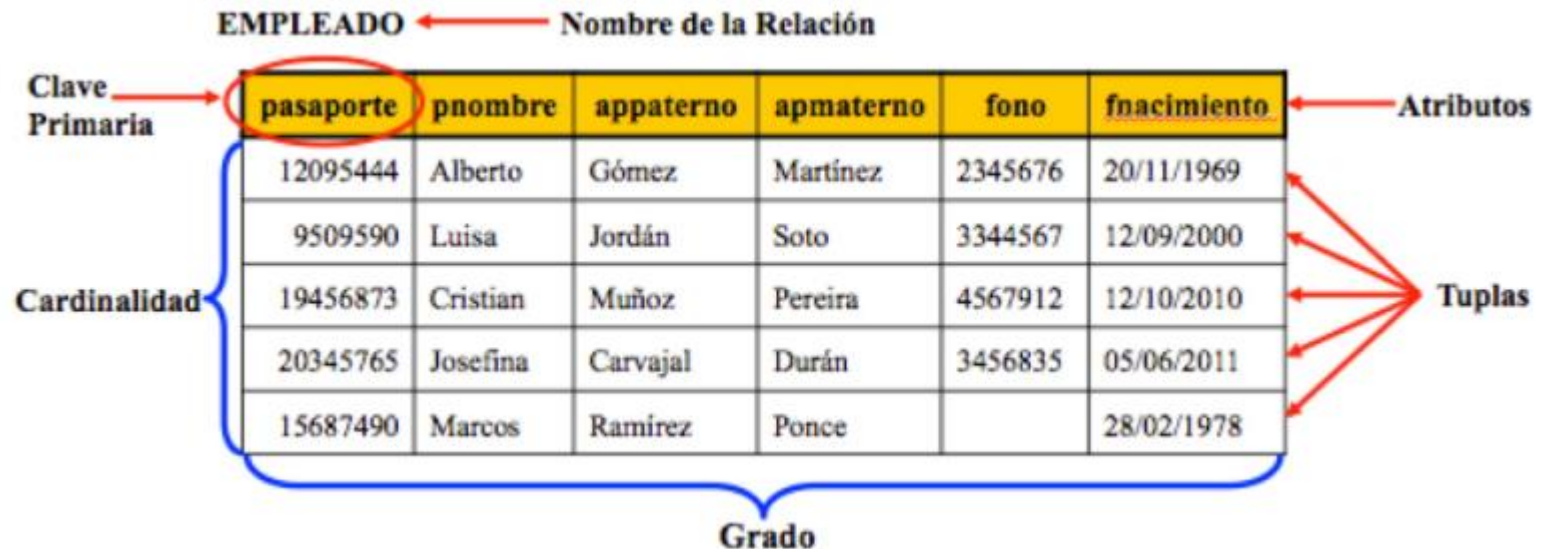
- **Relación:** Una relación es una tabla que representa una entidad del mundo real o una asociación entre entidades. Las relaciones están formadas por tuplas y atributos.
- **Atributo:** Es cada una de las columnas de una tabla. Cada atributo tiene un dominio, que es un conjunto de valores permitidos para ese atributo.
- **Dominio:** Es el conjunto de valores posibles que un atributo puede tomar. Por ejemplo, el atributo Género podría tener un dominio compuesto por los valores Masculino y Femenino.
- **Clave Primaria (Primary Key):** Un conjunto de uno o más atributos que identifican de manera única cada tupla en una relación. Es un identificador único y no puede contener valores nulos.
- **Clave Foránea (Foreign Key):** Un atributo o conjunto de atributos en una relación que refiere a una clave primaria en otra relación, estableciendo una relación entre las dos tablas.

Teoría: Reglas de integridad del modelo relacional

- **Integridad de Entidad:** Asegura que cada tabla en la base de datos tenga una clave primaria que sea única y no nula. Esto garantiza que cada registro sea único y no exista ambigüedad en la identificación de registros.
- **Integridad Referencial:** Establece que las claves foráneas en una tabla deben corresponder a valores existentes en la tabla referenciada. Esta regla asegura que las relaciones entre tablas sean coherentes y previene la existencia de referencias inválidas.
- **Integridad de Dominio:** Asegura que los valores de un atributo deben pertenecer al dominio específico de ese atributo. Esto previene la entrada de datos que no sean válidos o que no tengan sentido dentro del contexto de la base de datos.
- .

Teoría: Otras definiciones modelo relacional

- **Cardinalidad:** número de tuplas(m).
- **Grado:** número de atributos(n).



Teoría: Relaciones entre tablas

- **Uno a uno (1:1)**
- **Uno a muchos (1:n)**
- **Muchos a muchos (n:m)**

Teoría: Modelo de dato no relacional

- NoSQL, es un enfoque utilizado en el diseño de bases de datos que permite el almacenamiento y consulta de datos fuera de las estructuras tradicionales que se encuentran en las bases de datos relacionales.

Ejemplos :

- Documento JSON
- XML
- Redes sociales
- Análisis de Big Data

Bases de datos relacionales y no relacionales

<https://youtu.be/xkMDTulV8r4>

Ejercicio de clase

Una base de datos relacional, entregar con el trabajo de cierre

Antes de Concluir la Sesión

- ¿Qué aprendí hoy? Explicar con el modelo de datos que desarrollé en clase
- Como lo puedo aplicar ?

Tarea para el Viernes Próximo

Ventajas y desventajas de los modelos de bases de datos vistos en clase.

Curso: Bases de Datos I

Ingeniería en Sistemas y Ciencias de la Computación

Clase 3: Proceso de diseño de Bases de Datos

Catedrático: Claudia Alegre

31 de enero de 2026

Dinámica de Introducción

Un sistema académico de la universidad.

¿Qué nos interesa registrar?

Tema

Proceso de Diseño de Bases de Datos

Indicadores de Logro

- Los estudiantes deben ser capaces de describir y los principios y técnicas de diseño de bases de datos, incluyendo el análisis de requerimientos.
- Los estudiantes serán capaces de modelar entidades de forma conceptual.

Actividades

- Explicación de la teoría.
- Ejemplos
- Validación de conocimientos

Teoría: Proceso de Diseño de Bases de datos

Ya tenemos los requerimientos, ahora vamos a diseñar...

En el proceso de diseño de una base de datos encontramos tres fases:

- Diseño Conceptual
- Diseño Lógico
- Diseño Físico

Teoría: Modelo Conceptual

La primera y más abstracta capa en el proceso de diseño.

Su propósito principal es ofrecer una vista simplificada y de alto nivel de la estructura de la base de datos, enfocándose en las principales entidades de datos relevantes para el negocio y las relaciones entre ellas.

Responde a la pregunta ¿Qué información debe almacenarse y por qué?

A menudo se representa mediante un Diagrama Entidad-Relación

Teoría: Modelo Conceptual

Practiquemos, démosle forma al sistema académico.

- Entidades
- Relaciones

Teoría: Modelo Lógico de Datos

Tomamos el marco conceptual y añadimos un nivel significativo de detalle, aún es independiente de la tecnología de base de datos específica que utilizaremos para implementarlo.

Definimos las entidades y sus atributos

Llaves primarias

Llaves foráneas

Relaciones entre entidades

Tipos de datos para los atributos

Teoría: Modelo Lógico de datos

Practiquemos, continuemos con el modelo para nuestro sistema académico.

- Que hace únicos los registros en las entidades: Llaves primarias
- Cómo relaciono las entidades: Llaves foráneas
- Tipos de Relaciones entre entidades
- Definamos los tipos de datos para los atributos

Teoría: Modelo Físico de Datos

La etapa final del proceso de modelado de datos. Traducimos el diseño lógico a las especificaciones técnicas necesarias para implementar la base de datos en un DBMS específico.

- Entidades se convierten en tablas
- Atributos se convierten en columnas
- Relaciones se convierten en restricciones de llave foránea
- Definición de tipos exactos de datos
- Indices
- Procedimientos almacenados
- Triggers
- Particiones de datos

Ejercicio de clase

Entrega tu Modelo Entidad relación completo

Tarea para el Viernes Próximo

Definir requerimientos para un modelo de datos de una aerolínea

Curso: Bases de Datos I

Ingeniería en Sistemas y Ciencias de la Computación

Clase 4: Normalización de Tablas

Catedrático: Claudia Alegre

7 de febrero de 2026

Dinámica de Introducción

Por alumno:

Hacer una lista general que contenga datos:

Alumno , dirección alumno, curso, créditos de curso, semestre curso, catedrático, teléfono catedrático, aula

Agrupemos datos:

- Alumnos
- Curso
- Catedrático
- Inscripción

Tema

Normalización de tablas

- Principios de normalización de tablas
- Formas normales (1FN, 2FN, 3FN)

Indicadores de Logro

- Los estudiantes serán capaces de describir y aplicar los principios de normalización de tablas, incluyendo las formas normales (1FN, 2FN, 3FN)
- Los estudiantes serán capaces de utilizar las técnicas para eliminar la redundancia y dependencias funcionales.

Actividades

- Explicación de la teoría.
- Ejemplos
- Validación de conocimientos

Teoría: Normalización de bases de datos

Proceso de diseño de bases de datos que organiza los datos en estructuras de tablas específicas.

Ayuda a mejorar la integridad de los datos, prevenir anomalías en los datos, minimizar la redundancia de datos y reforzar el rendimiento de las consultas.

Teoría: Formas normales

Conjuntos de reglas que rigen cómo se organizan los atributos dentro de una tabla.

Estas reglas se basan en gran medida en las relaciones entre atributos (columnas), incluidas las claves utilizadas para identificar filas de forma única.

Teoría: Beneficios de normalización de bases de datos..

- Prevención de anomalías de datos

Cuando las tablas más grandes y complejas se descomponen (o dividen) en tablas más pequeñas y sencillas, modificar una base de datos se convierte en un proceso más fácil y menos propenso a errores, y limita los cambios a las tablas ahora más pequeñas de datos relacionados.

- Reducción de la redundancia de datos involuntaria

Aunque la redundancia de datos intencionada puede ayudar a mejorar la calidad de los datos, la seguridad y la disponibilidad, la redundancia no intencionada es el efecto de los sistemas que crean datos duplicados de forma inadvertida, lo que da lugar a ineficiencias.

Teoría: ...Beneficios de normalización de bases de datos

- Ahorro de costos de almacenamiento de datos

Reducir los datos duplicados mediante la normalización de la base de datos puede reducir los costos de almacenamiento de datos. Esto es especialmente importante para entornos de nube donde los precios a menudo se basan en el volumen de almacenamiento de datos utilizado.

- Recuperación de datos más rápida

Una menor redundancia de datos debido a la normalización también puede conducir a consultas de datos más rápidas, ya que una menor redundancia a menudo requiere menos procesamiento de datos durante las búsquedas.

Teoría: Las llaves

- Principal
- Compuesta
- Candidata
- Foránea
- Super-clave

Una superclave es cualquier conjunto de atributos que permite identificar de manera única cada fila (tupla) en una tabla.

No necesariamente es la clave más “eficiente” o mínima, simplemente garantiza unicidad.

Todas las claves candidatas son superclaves, pero no todas las superclaves son candidatas.

Actividad: Ejercicio

Identificar Llaves:

- Principal
- Compuesta
- Candidata
- Foránea
- Super-clave

ISBN	Título	Autor_ID	Editorial
978-1234567	Bases de Datos	1	Alfa
978-7654321	Redes de Computadoras	2	Beta

Autor_ID	Nombre_Autor	Correo
1	Ana López	ana@correo.com
2	Luis Pérez	luis@correo.com

Prestamo_ID	ISBN	Usuario_ID	Fecha_Prestamo
1001	978-1234567	U001	01-02-2026
1002	978-7654321	U002	02-02-2026

Usuario_ID	Nombre_Usuario	Correo_Usuario
U001	Marta Díaz	marta@correo.com
U002	Carlos Gómez	carlos@correo.com

Tabla Libros

Tabla Autores

Tabla Prestamos

Tabla Usuarios

Teoría: Formas normales

- 1ra forma normal
- 2da forma normal
- 3ra forma normal y Boyce Codd
- 4ta forma normal
- 5ta forma normal

Teoría: Primera forma normal

Para una tabla ser la primera forma normal, debe cumplir el siguiente criterio:

- Una sola celda no debe contener más de un valor (atomicidad)
- Debe haber una clave primaria para identificación
- No filas o columnas duplicadas
- Cada columna debe tener solamente un valor por cada fila en la tabla

Teoría: Primera forma normal

Tabla no normalizada

IDPedido	Cliente	Productos
1	Juan	Manzanas, Plátanos, Naranjas
2	Alice	Uvas, Fresas
3	Bob	Limonas, Limas

Tabla normalizada

IDPedido	Cliente	Producto
1	Juan	Manzanas
1	Juan	Plátanos
1	Juan	Naranjas
2	Alice	Uvas
2	Alice	Fresas
3	Bob	Limonas
3	Bob	Limas

Teoría: Segunda forma normal

En 1FN solamente elimina los grupos repetitivos, no la redundancia. Por eso hay 2FN.

Una tabla se dice que está en 2FN si cumple el siguiente criterio:

- ya está en 1FN
- no tiene dependencia parcial. Es decir, todos los atributos no claves son totalmente dependientes de la clave primaria

Teoría: Segunda forma normal

Esta tabla viola la 2NF porque el Instructor atributo depende de ambos IDEstudiante y IDCurso.

IDEstudiante	IDCurso	NombreCurso	Instructor
1	101	Matemáticas	Prof. Smith
1	102	Física	Prof. Johnson
2	101	Matemáticas	Prof. Smith
3	103	Historia	Prof. Davis

Para lograr la 2NF, dividimos la tabla en dos tablas separadas:

Ahora, el Instructor atributo depende únicamente del CourseID, y la tabla está en 2FN.

IDEstudiante	NombreEstudiante
1	John
2	Alice
3	Bob

Tabla de Cursos:

IDCurso	NombreCurso	Instructor
101	Matemáticas	Prof. Smith
102	Física	Prof. Johnson
103	Historia	Prof. Davis

Teoría: Tercera forma normal

Cuando una tabla está en 2FN, elimina los grupos repetitivos y la redundancia, pero no elimina la dependencia parcial transitiva.

Esto significa que un atributo no principal (un atributo que no forma parte de la clave del candidato) es dependiente de otro atributo no principal. Esto es lo que la tercera forma normal (3FN) elimina.

Para que una tabla esté en 3FN, debe

- estar en 2FN
- no tiene dependencia parcial transitiva

Teoría: Tercera forma normal

Esta tabla viola la 3FN porque el Gerente atributo depende de IDEmpleado, no directamente de la clave primaria.

IDEmpleado	IDProyecto	NombreProyecto	Gerente
1	101	ProyectoA	John
1	102	ProyectoB	Alice
2	101	ProyectoA	John
3	103	ProyectoC	Bob

Tabla de Empleados:

IDEmpleado	NombreEmpleado
1	Juan
2	Alice
3	Bob

Tabla de Proyectos:

IDProyecto	NombreProyecto
101	ProyectoA
102	ProyectoB
103	ProyectoC

Tabla de EmpleadosProyectos:

IDEmpleado	IDProyecto
1	101
1	102
2	101
3	103



Para llevarlo a la 3FN, dividimos la tabla en dos tablas separadas
Ahora, el Gerente atributo depende del ID del Proyecto, y la tabla está en 3FN.

Teoría: Formas normales

- Boyce Codd
- 4ta forma normal
- 5ta forma normal

https://youtu.be/stX4Ka_6au8

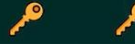
Teoría: Formas normales



AUTOR	TITULO	PUBLICACION	EDITORIAL	EDITOR
A	Libro 1	2004	Anagrama	Luis
B	Libro 2	2025	Planeta	Martha
B	Libro 3	2000	Anagrama	Luis
B	Libro 4	1985	Planeta	Hector
C	Libro 5	1994	Planeta	Samuel
C	Libro 6	2020	Anagrama	Luis

EDITORIAL -> EDITOR

Boyce Codd



AUTOR	TITULO	PUBLICACION	EDITORIAL
A	Libro 1	2004	Anagrama
B	Libro 2	2025	Planeta
B	Libro 3	2000	Anagrama
B	Libro 4	1985	Planeta
C	Libro 5	1994	Planeta
C	Libro 6	2020	Anagrama

PUBLICACION,EDITORIAL-> AUTOR,TITULO



EDITOR	EDITORIAL
Luis	Anagrama
Martha	Planeta
Hector	Planeta
Samuel	Planeta

EDITORIAL-> EDITOR

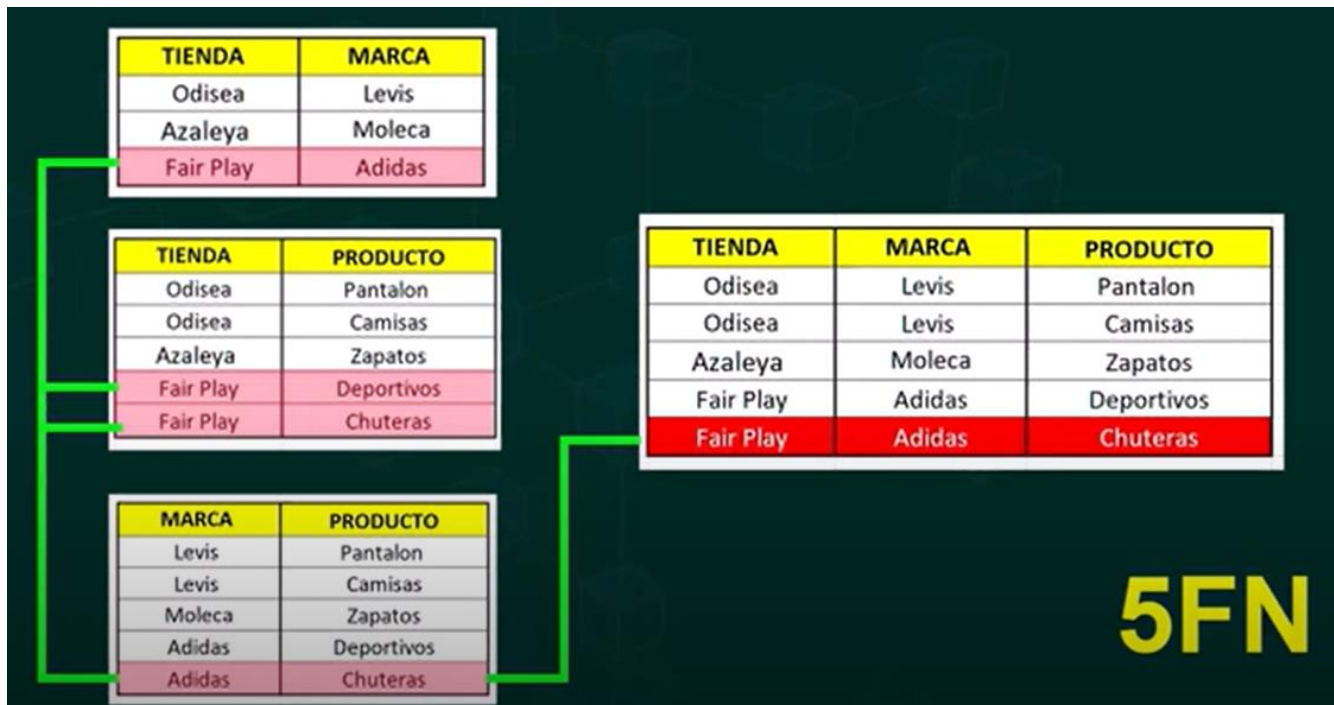
Teoría: Formas normales

RESTAURANTES		
RESTAURANTE	TIPO DE PIZZA	ZONA DE ENTREGA
Italia	Napolitana	Sur
Mr Pizza	Romana	Central
Dominos	Hawaiana	Norte
Napoles	Romana	Sur
Steve	Hawaiana	Norte
Viandante	Romana	Central

4ta forma normal

Variedades por restaurante		Zonas de entrega por restaurante	
RESTAURANTE	TIPO DE PIZZA	RESTAURANTE	ZONA DE ENTREGA
Italia	Napolitana	Italia	Sur
Mr Pizza	Romana	Mr Pizza	Central
Dominos	Hawaiana	Dominos	Norte
Napoles	Romana	Napoles	Sur
Steve	Hawaiana	Steve	Norte
Viandante	Romana	Viandante	Central

Teoría: Formas normales



Ejercicio de clase

Entregar el ejercicio realizado en clase en tercera forma normal

Pedido_ID	Cliente	Teléfono	Productos	Cantidades	Mesero
001	Ana	555-1111	Café, Croissant, Jugo	2, 1, 1	Luis
002	Marta	555-2222	Sandwich, Café	1, 1	Carlos

Ejercicio de clase2

Entregar el ejercicio realizado en clase en tercera forma normal

Empleado_ID	Nombre	Teléfonos	Proyectos	Fechas
001	Ana	555-1111, 555-2222	ERP, CRM	Ene-2026, Feb-2026
002	Luis	555-3333	App Móvil, Web Corporativa	Mar-2026, Abr-2026

Antes de Concluir la Sesión

- Explique las 3 formas normales vistas en clase

Tarea para el Viernes Próximo

¿Por que normalizar?

Curso: Bases de Datos I
Ingeniería en Sistemas y Ciencias de la
Computación

Clase 5: Diseño Diagramas Entidad Relación

Catedrático: Claudia Alegre

14 de febrero de 2026

Dinámica de Introducción

Por alumno

Hay un mercado gastronómico donde conviven varios actores:

Los chefs que preparan platillos únicos, los clientes que visitan el mercado buscando experiencias culinarias, los puestos de comida, cada uno especializado en un tipo de cocina (italiana, mexicana, japonesa), los ingredientes, que provienen de agricultores locales y se registran para garantizar calidad, los concursos gastronómicos, organizados cada mes para premiar al mejor platillo.

En este lugar cada cliente puede visitar varios puestos y probar distintos platillos, los chefs pueden trabajar en más de un puesto, pero cada platillo tiene un chef responsable, los ingredientes se compran a proveedores y se usan en varios platillos, los concursos registran qué platillos participan y quiénes fueron los clientes que votaron, los premios se entregan a los chefs ganadores y se registran en la historia del mercado.

Identifiquemos entidades, colocándolas en una lista

Tema

Diseño de diagramas entidad relación

Indicadores de Logro

- Los estudiantes serán capaces de describir y aplicar los principios de diseño de diagramas Entidad-Relación, incluyendo la representación de entidades, relaciones y atributos.
- Los estudiantes serán capaces de analizar un caso de uso y diseñar un diagrama ER para representarlo.

Actividades

- Explicación de la teoría.
- Ejemplos usemos Drawio.....
<https://www.drawio.com/>
- Validación de conocimientos

Teoría: Modelo entidad relación - Componentes

El modelo entidad-relación se compone de varios elementos clave que permiten su representación visual y su posterior implementación.

Entidades

Atributos

Relaciones

Teoría: Modelo entidad relación - Ventajas

- Proporciona una representación visual clara y concisa de la estructura de la base de datos, lo que facilita la comprensión por parte de todos los implicados en el proyecto
- Flexibilidad que brinda en el proceso de diseño, medida que se identifican nuevas necesidades o se realizan modificaciones en las tecnologías utilizadas, es relativamente sencillo ajustar o extender el modelo entidad-relación.
- Promueve la normalización de datos, lo que ayuda a evitar la redundancia y la inconsistencia, al asegurar que cada dato se almacene en un único lugar.

Caso: Modelo ER – Un caso

Sistema de Recursos Humanos

En un sistema de recursos humanos, se pueden identificar las siguientes entidades:

- **Empleado:** Persona que trabaja en la organización.
- **Departamento:** División dentro de la empresa que agrupa a empleados específicos.
- **Proyecto:** Tareas o iniciativas que requieren la participación de empleados.

Las relaciones que pueden establecerse son:

- Un **empleado** puede pertenecer a solo un **departamento**.
- Un **departamento** puede tener múltiples **empleados**.
- Un **empleado** puede trabajar en varios **proyectos**.

Enriquecer el modelo:

- Coordinador-empleado
- Dirección- gerencia – departamento
- Proyecto - Actividades

Caso: ER – Caso clínica

Una clínica que debe considerar:

1. Gestión de pacientes

Registro de datos personales (nombre, edad, contacto, historial médico).

Expediente clínico digital (consultas, diagnósticos, tratamientos, alergias).

Agenda de citas y recordatorios automáticos.

Seguimiento de evolución y resultados de laboratorio.

2. Gestión médica

Agenda de médicos (horarios, disponibilidad).

Registro de consultas y notas médicas.

Prescripción electrónica de medicamentos.

Referencias a especialistas o estudios adicionales.

3. Laboratorios y estudios

Solicitud y registro de análisis clínicos.

Integración con equipos de laboratorio para subir resultados.

Historial de estudios por paciente.

Antes de Concluir la Sesión

- ¿Que aprendimos hoy? Usa un modelo elaborado en clase para explicarlo.
- No olvides compartir tus modelos de datos

Tarea : ER – Caso tienda online

Ahora una tienda de pedidos online

Los clientes se registran con sus datos personales (nombre, correo, dirección, teléfono).

Cada cliente puede realizar varios pedidos, que incluyen fecha, estado (pendiente, enviado, entregado) y método de pago.

Los productos tienen atributos como nombre, categoría, precio y stock disponible. Los proveedores abastecen los productos, con información de contacto y condiciones de entrega.

Cada pedido puede incluir varios productos, y cada producto puede aparecer en distintos pedidos.

Los pagos se registran con monto, fecha y método (tarjeta, transferencia, PayPal).

La tienda ofrece reseñas, donde los clientes califican productos y dejan comentarios.

Tarea para el Viernes Próximo

Elabora modelo de datos para caso tienda online

Estudiar.....

Curso: Bases de Datos I

Ingeniería en Sistemas y Ciencias de la Computación

Clase 7: Creación de Tablas en base a Diseño Diagramas Entidad Relación

Catedrático: Claudia Alegre

28 de febrero de 2026

Dinámica de Introducción

Por alumno

“Sistema de Gestión de Cine y Entretenimiento”

La cadena de cines **CineMundo** quiere implementar un sistema digital para administrar sus operaciones.

Los **clientes** se registran con nombre, correo, teléfono y fecha de nacimiento.

Los clientes pueden comprar **boletos** para funciones de películas, indicando asiento, fecha y método de pago.

Las **películas** tienen título, género, duración, clasificación y director.

Cada película se proyecta en varias **funciones**, que incluyen fecha, hora y sala asignada.

Las **salas** tienen número, capacidad y tipo (2D, 3D, IMAX).

Los **empleados** del cine se dividen en roles: cajeros, acomodadores y técnicos de proyección.

El sistema debe registrar **proveedores externos** que suministran alimentos y bebidas para la dulcería, con nombre de empresa, contacto y tipo de producto.

Los clientes pueden dejar **reseñas** de películas, con calificación (1 a 5 estrellas) y comentario.

La administración necesita reportes automáticos:

- Películas más vistas.
- Clientes frecuentes.
- Ventas por sala y por película.

Dinámica de Introducción

Entidades posibles: Cliente, Boleto, Película, Función, Sala, Empleado, Proveedor, Reseña.

Atributos clave:

- Cliente (id_cliente, nombre, correo, teléfono, fecha_nacimiento)
- Boleto (id_boleto, asiento, fecha, método_pago, id_cliente, id_función)
- Película (id_pelicula, título, género, duración, clasificación, director)
- Función (id_función, fecha, hora, id_pelicula, id_sala)
- Sala (id_sala, número, capacidad, tipo)
- Empleado (id_empleado, nombre, rol, turno)
- Proveedor (id_proveedor, empresa, contacto, tipo_producto)
- Reseña (id_reseña, calificación, comentario, id_cliente, id_pelicula)

Dinámica de Introducción

Relaciones

Cliente–Boleto $\rightarrow (1:n)$

Boleto–Función $\rightarrow (n:1)$

Función–Película $\rightarrow (n:1)$

Función–Sala $\rightarrow (n:1)$

Película–Reseña $\rightarrow (1:n)$

Cliente–Reseña $\rightarrow (1:n)$

Proveedor–Sala $\rightarrow (n:m)$ (proveedores abastecen dulcerías de varias salas)

Empleado–Sala $\rightarrow (1:n)$

Dinámica de Introducción

Entidades y atributos

Cliente

id_cliente (PK)
nombre
correo
teléfono
fecha_nacimiento

Boleto

id_boleto (PK)
asiento
fecha
método_pago id_cliente (FK)
id_función (FK)

Película

id_pelicula (PK)
título
género
duración
clasificación
director

Función

id_función (PK)
fecha
Hora
id_pelicula (FK)
id_sala (FK)

Sala

id_sala (PK)
número
capacidad
tipo (2D, 3D, IMAX)

Empleado

id_empleado (PK)
nombre
rol (cajero, acomodador, técnico)
turno
id_sala (FK)

Proveedor

id_proveedor (PK)
empresa
contacto
tipo_producto

Reseña

id_reseña (PK)
calificación (1–5)
comentario
id_cliente (FK)
id_pelicula (FK)

Dinámica de Introducción

Empleado
• ID_Empleado (PK) • Nombre • Rol • ID_Sala (FK) • Turno

Boleto
• ID_Boleto (PK) • ID_Cliente (FK) • ID_Funcion (PK) • Asiento • Fecha • Método_Pago

Proveedor
• ID_Proveedor (PK) • Empresa • Contacto • Tipo_Producto

Sala
• ID_Sala (PK) • Número • Capacidad • Tipo

Cliente
• ID_Cliente (PK) • Nombre • Correo • Teléfono • Fecha_Nacimiento

Pelicula
• ID_Pelicula (PK) • Titulo • Género • Duracion • Clasificación • Director

Funcion
• ID_Funcion (PK) • Fecha • Hora • ID_Pelicula (FK) • ID_Sala (FK)

Reseña
• ID_Reseña (PK) • ID_Cliente (PK) • ID_Pelicula (FK) • Calificación • Comentario

Caso: Modelo ER – Un caso

*****Ahora en papel*****

Sistema de Recursos Humanos

En un sistema de recursos humanos, se pueden identificar las siguientes entidades:

- **Empleado:** Persona que trabaja en la organización.
- **Departamento:** División dentro de la empresa que agrupa a empleados específicos.
- **Proyecto:** Tareas o iniciativas que requieren la participación de empleados.

Las relaciones que pueden establecerse son:

- Un **empleado** puede pertenecer a solo un **departamento**.
- Un **departamento** puede tener múltiples **empleados**.
- Un **empleado** puede trabajar en varios **proyectos**.

Enriquecer el modelo:

- Coordinador-empleado
- Dirección- gerencia – departamento
- Proyecto - Actividades

Tema

Creación de Tablas para Diseño de diagramas entidad relación

Indicadores de Logro

- Los estudiantes deben ser capaces de escribir sentencias SQL para crear tablas en un *DBMS*, utilizando los diseños de diagramas ER y esquemas relacionales previamente elaborados, y asegurando la integridad de los datos mediante la creación de restricciones y claves primarias.

Actividades

- Explicación de la teoría
- Tipos de datos y definición de tablas
- Ejemplos

Teoría: Create table

La instrucción CREATE TABLE es el **comando principal del lenguaje de definición de datos (DDL)** que determina cómo se almacenan, organizan y acceden a tus datos.

Antes de escribirla, debes decidir qué representa la tabla, cómo se conecta con otras y cómo su estructura respaldará futuras consultas.

Teoría: Create table

```
CREATE TABLE schema_name.table_name (  
  column_name1 data_type constraint,  
  column_name2 data_type constraint,  
  column_name data_type constraint,  
  ...  
);
```

Teoría: Create table

```
CREATE TABLE schema_name.table_name (  
column_name1 data_type constraint,  
column_name2 data_type constraint,  
column_name data_type constraint,  
...  
);
```

- `schema_name`: Espacio de nombres opcional para organizar tablas.
- `table_name`: Nombre claro y descriptivo para la entidad.
- `column_name`: El nombre de un campo (atributo) de la tabla.
- `data_type`: Define el formato, como INT, VARCHAR o DATE.
- `constraint`: Reglas opcionales como PRIMARY KEY, NOT NULL o UNIQUE.

Teoría: Tipos de datos

Tipos de datos comunes	Tipos de datos del sistema de SQL Server.	Sinónimo ANSI	Número de bytes
Entero	int	<i>integer</i>	4
	bigint	–	8
	smallint, tinyint	–	2, 1
Numérico exacto	decimal[(p[, s])]	<i>dec</i>	2–17
	numeric[(p[, s])]	–	
Numérico aproximado	float[(n)]	<i>double precision</i> , <i>float[(n)]</i> para <i>n=8-15</i>	8
	real	<i>float[(n)]</i> para <i>n=1-7</i>	4
Moneda	money, smallmoney	–	8, 4
Fecha y hora	Datetime	–	8
	smalldatetime	–	4

Tipos de datos comunes	Tipos de datos del sistema de SQL Server.	Sinónimo ANSI	Número de bytes
Carácter	char[(n)]	<i>character[(n)]</i>	0–8000
	varchar[(n)]	<i>char VARYING[(n)]</i> <i>character</i>	
	text	<i>VARYING[(n)]</i> –	0 a 2 GB
Caracteres Unicode	nchar[(n)]	–	0–8000
	nvarchar[(n)]	–	(4000 caracteres)
	ntext	–	0 a 2 GB
Binario	binary[(n)]	–	0–8000
	varbinary[(n)]	<i>binary</i> <i>VARYING[(n)]</i>	
Imagen	image	–	0 a 2 GB
Identificador global	uniqueidentifier	–	16
Especial	bit, cursor, uniqueidentifier	–	1, 0–8
	timestamp	rowversion	8
	sysname	–	256
	table	–	
	sql_variant	–	0–8016

Tipos de datos...

Tipos de datos comunes en SQL Server

- **Numéricos**
- **Texto**
- **Fecha y hora**
- **Otros**

Tipo	Descripción
DATE	Solo fecha
TIME	Solo hora
DATETIME	Fecha y hora (hasta 3 ms)
DATETIME2	Fecha y hora con más precisión
SMALLDATETIME	Fecha y hora con precisión de 1 minuto

Tipo	Descripción
CHAR(n)	Texto fijo de longitud n
VARCHAR(n)	Texto variable hasta n caracteres
NVARCHAR(n)	Texto variable Unicode
TEXT / NTEXT	Texto largo (obsoleto, usar VARCHAR(MAX) o NVARCHAR(MAX))

Tipo	Descripción
BIT	Booleano (0 o 1)
BINARY(n)	Datos binarios fijos
VARBINARY(n)	Datos binarios variables
UNIQUEIDENTIFIER	GUID

Tipo	Descripción	Rango aproximado
INT	Entero	-2,147,483,648 a 2,147,483,647
BIGINT	Entero grande	±9 cuatrillones
SMALLINT	Entero pequeño	-32,768 a 32,767
TINYINT	Entero muy pequeño	0 a 255
DECIMAL(p,s) / NUMERIC(p,s)	Número exacto con precisión y escala	Depende de p y s
FLOAT	Número de punto flotante	Aproximado

Tipos de datos

- ✓ Si la longitud de la columna varía, utilice uno de los tipos de datos variables
- ✓ Use tinyint adecuadamente
- ✓ Para tipos de datos numéricos, use los decimales más frecuentes
- ✓ Si el almacenamiento es superior a 8000 bytes, utilice text o image
- ✓ Para la moneda utilice el tipo de datos money
- ✓ No utilice float y real como claves principales

EJEMPLO

-- Create orders table

```
CREATE TABLE orders (  
  order_id INT PRIMARY KEY,  
  customer_id INT NOT NULL, -- Link order → customer  
  order_date TIMESTAMP DEFAULT NOW(),  
  total_amount DECIMAL(10,2) NOT NULL,  
  FOREIGN KEY (customer_id) REFERENCES customers(customer_id)  
  -- Enforce relationship and ensure customer exists  
);
```

Para crear tablas...

- **Normalización excesiva:** Dividir las tablas en tantos fragmentos pequeños puede complicar las uniones y ralentizar las consultas, mientras que una normalización insuficiente puede provocar redundancia de datos. Para evitarlo, utiliza siempre 3NF inicialmente y, a continuación, utiliza la desnormalización solo para tablas de informes específicas en las que priorices la velocidad de lectura sobre la simplicidad de actualización.
- **Diseño deficiente de las restricciones:** Si no defines tus restricciones, como las claves externas que faltan, puedes acabar con registros huérfanos o datos incoherentes que son difíciles de limpiar. Especifica siempre un índice en cada columna que sirva de FOREIGN KEY a para garantizar que las uniones sean rápidas.

...Para crear tablas

- **Denominación y documentación claras:** Utiliza siempre nombres de tablas y columnas predecibles y descriptivos. Mantén un registro de cambios sencillo para las actualizaciones del esquema y evita utilizar abreviaturas que los futuros programadores puedan no entender.
- **Permisos y consideraciones de seguridad:** Al diseñar tablas para entornos de producción, concede solo los privilegios necesarios, como los que permiten crear, modificar o eliminar tablas, y separa el acceso de lectura y escritura para proteger los datos críticos. Además, implementa controles a nivel de esquema y registros de auditoría para realizar un seguimiento de los cambios, lo que evita el acceso no autorizado y garantiza el cumplimiento normativo y la seguridad de los datos.

Mas ejemplos:

```
CREATE TABLE Clientes (  
  ClienteID INT PRIMARY KEY IDENTITY(1,1), -- Autoincremental  
  Nombre NVARCHAR(100) NOT NULL,  
  Email NVARCHAR(150) UNIQUE NOT NULL,  
  FechaRegistro DATETIME DEFAULT GETDATE(),  
  Activo BIT DEFAULT 1 );
```

Explicación:

- IDENTITY(1,1): Comienza en 1 y aumenta de 1 en 1.
- NVARCHAR: Texto que soporta caracteres Unicode (acentos, ñ, etc.).
- BIT: Booleano (0 o 1).
- GETDATE(): Fecha y hora actual.

Tarea para el Viernes Próximo

Investigar para que sirve sentencias Alter table y Drop table

Instalar SQL Server Express

Curso: Bases de Datos I

Ingeniería en Sistemas y Ciencias de la Computación

Clase 8: Lenguaje SQL: Almacenamiento, recuperación y mantenimiento de datos

Catedrático: Claudia Alegre

7 de marzo de 2026

Dinámica de Introducción

Hagamos un corto modelo de datos de un hospital, considerando:

- Pacientes que visitan el hospital
- Especialidades del hospital
- Visitas de pacientes a las especialidades

Tema

Lenguaje SQL: Almacenamiento, recuperación y mantenimiento de datos

Indicadores de Logro

- Los estudiantes serán capaces de escribir consultas SQL complejas y precisas para almacenar, recuperar y modificar datos en una base de datos.
- Los estudiantes serán capaces de escribir sentencias *INSERT*, *UPDATE* y *DELETE* para agregar, modificar y eliminar registros en una base de datos.

Actividades

1. Explicación de la teoría
2. Ejemplos
3. Ejercicios prácticos

Teoría: Insert, update y Delete

INSERT, UPDATE y DELETE, se conocen como declaraciones de lenguaje de manipulación de datos (DML), y permiten a los usuarios de SQL ver y gestionar los datos.

Teoría: Insert

Primero, necesitamos identificar la tabla en la que queremos insertar filas. A continuación, especificamos las columnas que queremos rellenar. Por último, estipulamos los valores que necesitamos añadir.

```
INSERT INTO nombreTabla  
(columna1, columna2, ...)  
VALUES (valor1, valor2, ...)
```

Teoría: Insert

Inserción de varias filas

Para insertar más de una fila de datos con una sola sentencia, utiliza paréntesis y comas para especificar las distintas filas nuevas.

```
INSERT INTO dbo.Paciente  
            (Apellido, Nombre, FechaNac, Sexo, [Peso], Altura, Vacuna)  
VALUES      ('Hitson', 'George', '2019-11-19', 'M', 13.9, 22.5, 'No'),  
VALUES      ('Hitson', 'Jenny', '2019-11-19', 'F', 13.7, 22.3, 'No')
```

Teoría: Insert

Recomendación:

Utiliza una transacción para probar tu inserción sin comprometer ni alterar tu tabla de manera permanente.

Comienza con **BEGIN TRANSACTION**, y finaliza la transacción con **COMMIT**, si deseas mantener los cambios, o **ROLLBACK**, si deseas revertir lo que has hecho.

```
BEGIN TRANSACTION
INSERT INTO dbo.Paciente
    (Apellido, Nombre, FechaNac, Sexo, [Peso], Altura, Vacuna)
VALUES
    ('Hitson', 'George', '2019-11-19', 'M', 13.9, 22.5, 'No'),
    ('Hitson', 'Jenny', '2019-11-19', 'F', 13.7, 22.3, 'No')

ROLLBACK
--COMMIT
```

Teoría: Update

La sentencia **UPDATE** se utiliza para cambiar el valor de una columna en una o varias filas.

Así es como se estructura generalmente una sentencia SQL **UPDATE**:

```
UPDATE nombreTabla  
SET columna1=valor1,  
columna2=valor2,...  
WHERE filtroColumna=filtroValor
```

Teoría: Update

El departamento de Emergencias se está cambiando a "Trauma y Cirugía de Emergencias", por lo que es necesario cambiar su nombre en la base de datos. Para actualizar este nombre, puedes ejecutar esta sentencia:

```
UPDATE dbo.Departamento  
    SET NombreDepartamento = 'Trauma y Cirugia de  
Emergencias'  
WHERE IDDepartamento = 3
```

Teoría: Delete

DELETE elimina una o varias filas de una tabla

```
UPDATE dbo.Departamento  
    SET NombreDepartamento = 'Trauma y Cirugia de  
Emergencias'  
WHERE IDDepartamento = 3
```

Teoría: Delete

```
INSERT INTO nappy  
  (contents, isFull)  
VALUES  
  ('poop', 1);  
  
DELETE FROM nappy  
WHERE contents = 'poop';  
  
DROP TABLE nappy; # into bin
```

Teoría: Delete

Sólo tiene dos partes: especificar la tabla y especificar **WHERE**, es decir, qué filas quieres eliminar.
Si conoces la clave primaria de una fila que desees eliminar, es mas sencillo

```
DELETE FROM dbo.Departamento  
WHERE IDDepartamento = 16
```


Practiquemos

¿Qué esperas?

INSERTA nuevos conocimientos a tu vida,
ACTUALIZA tu formación en SQL y
ELIMINA todas las excusas.

Antes de Concluir la Sesión

- Subir los ejercicios de clase

Tarea para el Viernes Próximo

Elabora un modelo ER para control académico de la USPG

Debemos llevar control de estudiantes, sus cursos, asignaciones, catedráticos y notas obtenidas (8 entidades mínimo)

Crear una base de datos

- Todos los campos llave deben ser numéricos de 10
- Todos los campos alfauméricos deben ser de longitud 75
- Los campos fecha deben llevar fecha-hora

Poblarla con al menos: 20 estudiantes, 5 cursos, 4 catedráticos, al menos 2 semestres de notas.

Envías los scripts de creación de todas las tablas involucradas y las inserciones de registros.

Curso: Bases de Datos I
Ingeniería en Sistemas y Ciencias de la
Computación

Clase 9:
Lenguaje SQL: Unión, Intersección, Except

Catedrático: Claudia Alegre

14 de marzo de 2026

Dinámica de Introducción

Una organización cultural quiere implementar un sistema para gestionar su festival anual de música. Actualmente todo se maneja en hojas de cálculo y correos, lo que genera confusión, considera los siguientes aspectos: Cada artista tiene un código único, nombre artístico, país de origen, género musical y contacto. Un artista puede participar en varios conciertos. Cada concierto tiene un código, nombre del evento, fecha, hora y escenario. Un concierto puede incluir varios artistas. Cada escenario tiene un código, nombre, capacidad y ubicación dentro del festival. Un escenario puede albergar varios conciertos, pero en horarios distintos. Cada entrada tiene un número, tipo (general, VIP), precio y estado (vendida, disponible). Una entrada corresponde a un concierto específico. Cada cliente tiene un código, nombre completo, correo electrónico y teléfono. Un cliente puede comprar varias entradas. Cada venta registra fecha, monto total y método de pago. Una venta puede incluir varias entradas y está asociada a un cliente.

- Elabora un modelo entidad relación de este caso, mínimo 6 entidades

Dinámica de Introducción

1. Artista

ID_Artista (PK)
Nombre_Artístico
País_Origen
Género_Musical
Contacto

2. Concierto

ID_Concierto (PK)
Nombre_Evento
Fecha
Hora
ID_Escenario (FK)

3. Escenario

ID_Escenario (PK)
Nombre
Capacidad
Pais_Escenario

4. Entrada

ID_Entrada (PK)
Tipo (General, VIP)
Precio
Estado (Disponible, Vendida)
ID_Concierto (FK)

5. Cliente

ID_Cliente (PK)
Nombre_Completo
Correo_Electrónico
Teléfono

6. Venta

ID_Venta (PK)
Fecha
Monto_Total
Método_Pago
ID_Cliente (FK)

7. Detalle_Venta (tabla intermedia para relacionar Ventas y Entradas)

ID_Detalle (PK)
ID_Venta (FK)
ID_Entrada (FK)

8. Participantes_Concierto

ID_Artista (PK)
ID_Concierto (PK)

Dinámica de Introducción

Ahora vamos a crear las tablas correspondientes al modelo y las vamos a poblar

Tema

Lenguaje SQL: Operaciones de conjuntos

Indicadores de Logro

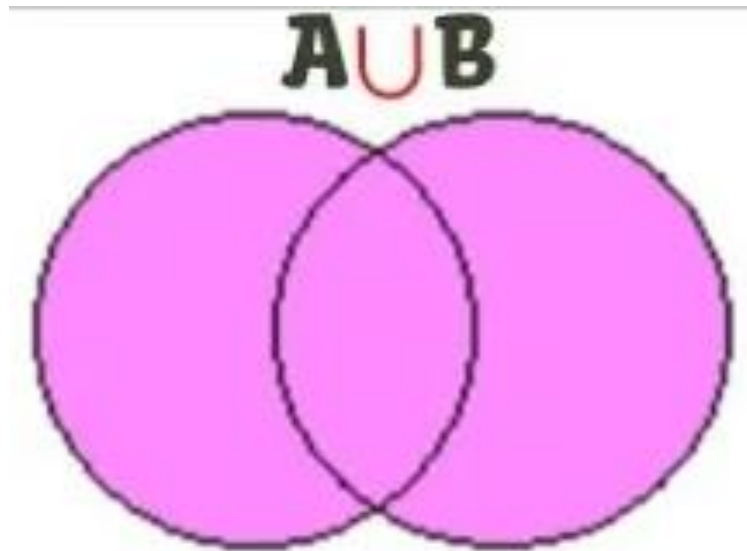
- Los estudiantes serán capaces de utilizar las operaciones de conjuntos (*UNION*, *INTERSECT* y *EXCEPT*) en consultas SQL para combinar y comparar
-

Actividades

1. Explicación de la teoría
2. Ejemplos
3. Ejercicios prácticos

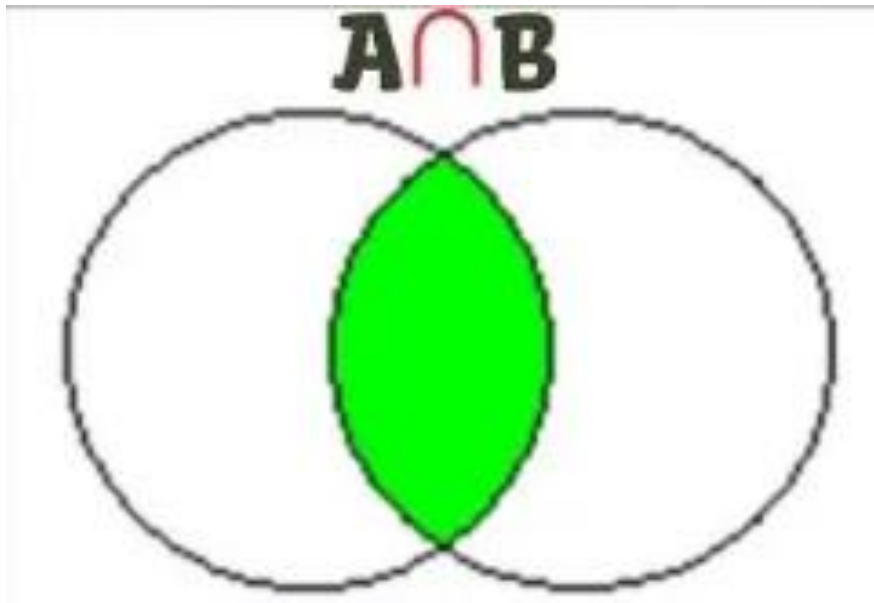
Teoría: Union, intersect, except

UNION



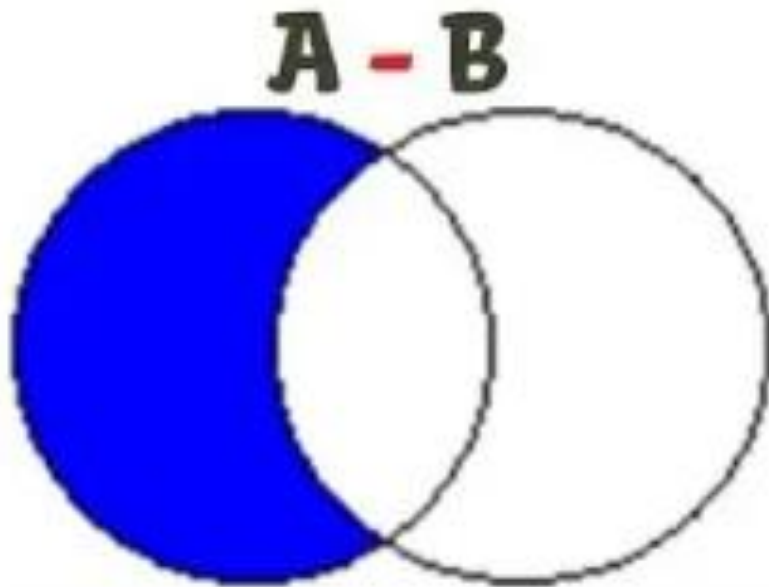
Teoría: Union, intersect, except

INTERSECCION



Teoría: Union, intersect, except

DIFERENCIA



Teoría: Union, intersect, except

UNION es una instrucción u operador que permite unir 2 o más SELECT concatenando los resultados de ejecución en un único conjunto de resultados.

UNION no crea registros individuales a partir de las columnas de las tablas que participan en la unión, sino que concatena el conjunto de resultados de las mismas.

Este funcionamiento está supeditado a unas reglas estrictas que no pueden ser violadas:

- ✓ Las **SELECT** que participen en el **UNION** tienen que constar del mismo número de columnas.
- ✓ Las columnas tienen que estar emparejadas en orden, siendo del mismo tipo de dato o en su defecto, datos compatibles, por ejemplo: **FLOAT** e **INT**...

UNION excluye valores duplicados. Si queremos que nuestro conjunto de resultados incluya duplicados, debemos sustituir el operador UNION por UNION ALL.

Teoría: Union, intersect, except

```
CREATE TABLE Empleados(  
  
    idEmpleado    INT IDENTITY(1,1),  
    nombre        VARCHAR(255) NOT NULL ,  
    apellidos     VARCHAR(255) NOT NULL ,  
    direccion     VARCHAR(255),  
    numeroEmpleado INT  
  
    CONSTRAINT pk_Empleados_idEmpleado  
PRIMARY KEY (idEmpleado)  
);
```

```
INSERT INTO Empleados  
    (nombre,  
    apellidos,  
    direccion,  
    numeroEmpleado)  
VALUES  
    ('Jesús',  
    'Hidalgo Galvez',  
    'C/ Conde Peñalver 24',  
    1001);
```

```
INSERT INTO Empleados  
    (nombre,  
    apellidos,  
    direccion,  
    numeroEmpleado)  
VALUES  
    ('Antonio',  
    'Silva García',  
    'C/ Aranda 1',  
    1002);
```

INSERTEMOS MAS REGISTROS...

.....Ahora crea otra tabla de Clientes, mismos tipos de datos en:

nombre, apellidos, dirección y cuenta

.. Inserta 10 registros..

Teoría: Union, intersect, except

```
SELECT nombre,  
       apellidos,  
       direccion,  
       cuenta as CuentaClienteNumeroEmpleado  
FROM Clientes  
WHERE cuenta IS NOT NULL  
UNION  
SELECT nombre,  
       apellidos,  
       direccion,  
       numeroEmpleado  
FROM Empleados  
ORDER BY CuentaClienteNumeroEmpleado ASC;
```

.....Ahora agrega UNION ALL

Teoría: Union, intersect, except

INTERSECT devuelve únicamente los registros comunes a la unión de todas las consultas

```
SELECT idClientes,  
        nombre,  
        apellidos,  
        direccion,  
        cuenta  
FROM Clientes  
WHERE idClientes IN (1,2,3,4)  
INTERSECT  
SELECT idClientes,  
        nombre,  
        apellidos,  
        direccion,  
        cuenta  
FROM Clientes  
WHERE idClientes IN (4,5,6,7);
```

Teoría: Union, intersect, except

EXCEPT devuelve únicamente los registros exclusivos de la primera consulta, no coincidentes con el resto:

```
SELECT idClientes,  
       nombre,  
       apellidos,  
       direccion,  
       cuenta  
FROM Clientes  
WHERE idClientes IN (1,2,3,4)  
EXCEPT  
SELECT idClientes,  
       nombre,  
       apellidos,  
       direccion,  
       cuenta  
FROM Clientes  
WHERE idClientes IN (4,5,6,7);
```

Practiquemos

Qué artistas participaron en conciertos? - INTERSECT

Elige un Artista e indica en que concierto participó? INTERSECT

Qué clientes NO han comprado entradas? - EXCEPT

Qué entradas están disponibles? - EXCEPT

Qué países representan los artistas o tienen un escenario - UNION

Antes de Concluir la Sesión

- Subir los scripts de los ejercicios de clase

Tarea para el Viernes Próximo

La Universidad organiza el **Torneo InterFacultades**, donde equipos de distintas carreras compiten en deportes como fútbol, baloncesto y voleibol. El evento es muy esperado, pero hasta ahora la información se maneja en hojas sueltas y correos, lo que genera confusión.

El comité organizador quiere un sistema que permita:

- Registrar a los **equipos** que representan a cada facultad.
- Identificar a los **jugadores** que forman parte de cada equipo.
- Llevar control de los **partidos** jugados, con sus resultados.
- Saber qué equipos aún no han jugado, cuáles ya compitieron y quiénes ganaron.

Entregar:

Modelo de datos en gráfica

Create tables

Insert (poblar al menos 6 equipos, 20 jugadores, 5 partidos)

Scripts utilizando únicamente selects simples y operaciones de unión, intersect y except respondiendo lo siguiente:

- *El comité quiere ver todos los equipos de Fútbol y Baloncesto en una sola lista.*
- *Encuentra los jugadores que pertenecen a equipos de la Facultad de Ingeniería y que tienen más de 20 años.*
- *Muestra los equipos de Voleibol que aún no han jugado ningún partido.*

Curso: Bases de Datos I
Ingeniería en Sistemas y Ciencias de la
Computación

Clase 10:
Lenguaje SQL: Agrupaciones y filtros en consultas

Catedrático: Claudia Alegre

21 de marzo de 2026

Dinámica de Introducción

El Museo Nacional de Arte y Cultura recibe visitantes de distintas ciudades y países. Cada visitante compra una entrada que puede ser General, Estudiante o VIP. El precio varía según el tipo de entrada, y la fecha de compra queda registrada para análisis de ingresos.

El museo organiza exhibiciones en diferentes salas: pintura, escultura, fotografía y piezas de historia natural. Cada exhibición tiene un nombre, un tipo, una sala asignada y un período de duración (fecha de inicio y fecha de fin). Durante su visita, los visitantes recorren una o varias exhibiciones. El museo registra cuánto tiempo pasa cada visitante en cada exhibición, lo que permite analizar la popularidad de las salas y el interés en los distintos tipos de arte.

Además, el museo cuenta con guías que acompañan a los visitantes en recorridos especiales, eventos culturales (conciertos, conferencias, talleres) que se realizan en paralelo a las exhibiciones y una tienda del museo, donde los visitantes pueden comprar recuerdos, libros y catálogos.

Elabora un modelo de datos entidad relación para este caso

Dinámica de Introducción

Visitantes

Personas que llegan al museo.

Datos: nombre, apellido, ciudad, país, fecha de visita.

Entradas

Registro de boletos comprados.

Datos: tipo (General, Estudiante, VIP), precio, fecha de compra, visitante asociado.

Exhibiciones

Muestras artísticas o culturales.

Datos: nombre, tipo (Pintura, Escultura, Fotografía, Historia Natural), sala, fechas de inicio y fin.

Visitas_Exhibiciones

Relación entre visitantes y exhibiciones.

Datos: visitante, exhibición, duración de la visita.

Eventos

Actividades culturales adicionales.

Datos: nombre, tipo (Conferencia, Taller, Concierto), fecha, costo.

Tienda

Ventas realizadas en la tienda del museo.

Datos: producto, categoría (Libro, Catálogo, Recuerdo), precio, cantidad, visitante asociado.

Guías

Personal que acompaña a los visitantes en recorridos especiales.

Datos: nombre, idioma, especialidad, horario asignado.

Salas

Espacios físicos donde se ubican las exhibiciones y eventos.

Datos: nombre, capacidad, ubicación (Planta baja, Segundo nivel, Terraza).

Dinámica de Introducción

Visitantes

- PK:** visitante_id
- Atributos: nombre, apellido, ciudad, país, fecha_visita

Entradas

- PK:** entrada_id
- FK:** visitante_id → Visitantes
- Atributos: tipo (General, Estudiante, VIP), precio, fecha_compra

Exhibiciones

- PK:** exhibicion_id
- FK:** sala_id → Salas
- Atributos: nombre, tipo (Pintura, Escultura, Fotografía, Historia Natural), fecha_inicio, fecha_fin

Visitas_Exhibiciones

- PK:** visita_id
- FK:** visitante_id → Visitantes
- FK:** exhibicion_id → Exhibiciones
- Atributos: duración_minutos

Eventos

- PK:** evento_id
- FK:** sala_id → Salas
- Atributos: nombre, tipo (Conferencia, Taller, Concierto), fecha, costo

Tienda

- PK:** venta_id
- FK:** visitante_id → Visitantes
- Atributos: producto, categoría (Libro, Catálogo, Recuerdo), precio, cantidad

Guías

- PK:** guia_id
- Atributos: nombre, idioma, especialidad, horario_asignado

Salas

- PK:** sala_id
- Atributos: nombre, capacidad, ubicación (Planta baja, Segundo nivel, Terraza)

Tema

Lenguaje SQL: Agrupaciones y filtros en consultas

Indicadores de Logro

- Los estudiantes serán capaces de utilizar la cláusula *WHERE* en consultas SQL para filtrar y seleccionar datos específicos de una tabla.
- Los estudiantes serán capaces de utilizar la cláusula *GROUP BY* en consultas SQL para agrupar datos y calcular funciones de agregación sobre ellos.
-

Actividades

1. Explicación de la teoría
2. Ejemplos
3. Ejercicios prácticos

Teoría: Where

La Cláusula WHERE es una herramienta fundamental en SQL que permite filtrar y seleccionar datos específicos de una tabla. Es una cláusula que se utiliza en conjunto con la sentencia SELECT y su función principal es establecer una condición que debe cumplirse para que los datos sean seleccionados y mostrados en la consulta.

Utilizada para seleccionar los datos que cumplen con ciertas condiciones, lo que facilita la búsqueda y el análisis de la información.

Teoría: Where

Ejemplo:

```
SELECT * FROM clientes WHERE fecha_compra > '2021-01-01'
```

Esta consulta mostrará todos los clientes que han realizado una compra después del 1 de enero de 2021.

Teoría: Where

También se puede utilizar con otras sentencias SQL, como UPDATE o DELETE, para modificar o eliminar sólo los registros que cumplen con ciertas condiciones.

Por ejemplo:

```
DELETE FROM clientes WHERE fecha_compra < '2020-01-01'
```

Esta sentencia eliminará todos los clientes que no hayan realizado una compra en los últimos dos años.

Ejercicio

Crear los objetos del modelo de datos del Museo

Insertar registros, al menos 10 elementos en 8 entidades

Teoría: Unir tablas

El principio básico de un unir tablas (joins) puede entenderse mejor si partimos de las operaciones que la componen. El fundamento de los joins son las siguientes operaciones de álgebra relacional:

- **Producto cartesiano**
- **Selección**

Teoría: Unir tablas

El **producto cartesiano** es una operación de la teoría de conjuntos en la que dos o más conjuntos se combinan entre sí. En el modelo de base de datos relacional se utiliza el producto cartesiano para interconectar conjuntos de tuplas en la forma de una tabla. El resultado de esta operación es otro conjunto de tuplas ordenadas, donde cada tupla está compuesta por un elemento de cada conjunto inicial.

En álgebra relacional se utiliza el **símbolo de la multiplicación ($\times$)** como operador para el producto cartesiano.

- Veamos un ejemplo:
- El producto cartesiano $A \times B$ de ambos conjuntos $A = \{x, y, z\}$ y $B = \{1, 2, 3\}$ es:
- $A \times B = \{(x,1), (x,2), (x,3), (y,1), (y,2), (y,3), (z,1), (z,2), (z,3)\}$

Teoría: Unir tablas

The diagram illustrates the Cartesian product of two sets, A and B, resulting in a new set $A \times B$.

Set A is represented by a vertical column of elements: x , y , and z .

Set B is represented by a horizontal row of elements: 1 , 2 , and 3 .

The resulting set $A \times B$ is a 3x3 grid of ordered pairs, where each element in A is paired with each element in B:

	1	2	3
x	$(x,1)$	$(x,2)$	$(x,3)$
y	$(y,1)$	$(y,2)$	$(y,3)$
z	$(z,1)$	$(z,2)$	$(z,3)$

Teoría: Selección

Por **selección** se entiende la operación del álgebra relacional que permite seleccionar unas tuplas determinadas de un conjunto inicial y mostrarlas como conjunto de resultados. Para establecer qué tuplas constituirán este conjunto se utiliza una expresión relacional. El resultado de la selección es, por lo tanto, un conjunto de tuplas que cumplen la **condición de la selección** definida en esta expresión relacional.

En el lenguaje de bases de datos SQL se definen las condiciones de selección con ayuda del comando WHERE.

```
SELECT * FROM Empleados WHERE Empleados.coche_ID = 1;
```

Teoría: Unión de tablas

Combinación de producto cartesiano y selección

- Los diferentes tipos de Joins combinan el producto cartesiano con una condición de selección

Teoría: Unión de tablas

Ejemplo: (A) Consideremos los elementos de la tabla EMPLEADO:

```
SELECT *  
FROM empleado
```

Nombre	Apellido1	Apellido2	Dni	FechaNac	Dirección	Sexo	Sueldo	SuperDni	Dno
José	P	Pérez	123456789	1965-09-01	Eloy I, 98	H	30000.00	333445555	5
Alberto	C	Sastre	333445555	1955-12-08	Avda Rios, 9	H	40000.00	888665555	5
Alicia	J	Celaya	999887777	1968-05-12	Gran Vía, 38	M	25000.00	987654321	4
Juana	S	Oreja	987654321	1941-06-20	Cerquillas, 76	M	43000.00	888665555	4
Fernando	O	Ordóñez	666884444	1962-09-15	Portillo, s/n	H	38000.00	333445555	5
Aurora	O	Avazueta	453453453	1972-07-31	Antón, 6	M	25000.00	333445555	5
Luis	P	Moreira	987987987	1969-03-29	Enebras, 90	H	25000.00	987654321	4
Eduardo	O	Paredes	888665555	1937-11-10	Las Peñas, 1	H	55000.00	0	1

Observa que tiene un total de 8 filas

(B) Consideremos ahora los elementos de la tabla DEPARTAMENTO:

```
SELECT *  
FROM departamento
```

NombreDpto	NumeroDpto	DniDirector	FechaIngresoDirector
Investigación	5	333445555	1988-05-22
Administración	4	987654321	1995-01-01
Sede Central	1	888665555	1981-06-19

Observa que tiene un total de 3 filas

Teoría: Unión de tablas

(C) Consideremos el **producto cartesiano** de ambas tablas:

```
SELECT *
FROM empleado, departamento
```

Nombre	Apellido1	Apellido2	Dni	FechaNac	Dirección	Sexo	Sueldo	SuperDni	Dpto	NombreDpto	NumeroDpto	DniDirector	FechaIngresoDirector
José	P	Pérez	123456789	1989-09-01	Eloy I. 90	H	30000.00	030445000	5	Base Central	1	000000000	1901-09-19
José	P	Pérez	123456789	1989-09-01	Eloy I. 90	H	30000.00	030445000	5	Administración	4	987004321	1999-01-01
José	P	Pérez	123456789	1989-09-01	Eloy I. 90	H	30000.00	030445000	5	Investigación	5	033445000	1999-05-22
Alberto	C	Basile	033445000	1989-12-08	Audix Ríos, 9	H	40000.00	000000000	5	Base Central	1	000000000	1901-09-19
Alberto	C	Basile	033445000	1989-12-08	Audix Ríos, 9	H	40000.00	000000000	5	Administración	4	987004321	1999-01-01
Alberto	C	Basile	033445000	1989-12-08	Audix Ríos, 9	H	40000.00	000000000	5	Investigación	5	033445000	1999-05-22
Alicia	J	Delayo	999997777	1999-05-12	Gran Vía, 38	M	25000.00	987954321	4	Base Central	1	000000000	1901-09-19
Alicia	J	Delayo	999997777	1999-05-12	Gran Vía, 38	M	25000.00	987954321	4	Administración	4	987004321	1999-01-01
Alicia	J	Delayo	999997777	1999-05-12	Gran Vía, 38	M	25000.00	987954321	4	Investigación	5	033445000	1999-05-22
Juana	S	Díaz	987954321	1941-05-20	Carquillas, 76	M	43000.00	000000000	4	Base Central	1	000000000	1901-09-19
Juana	S	Díaz	987954321	1941-05-20	Carquillas, 76	M	43000.00	000000000	4	Administración	4	987004321	1999-01-01
Juana	S	Díaz	987954321	1941-05-20	Carquillas, 76	M	43000.00	000000000	4	Investigación	5	033445000	1999-05-22
Fernando	O	Ordóñez	000004444	1982-09-15	Portillo, s/n	H	36000.00	030445000	5	Base Central	1	000000000	1901-09-19
Fernando	O	Ordóñez	000004444	1982-09-15	Portillo, s/n	H	36000.00	030445000	5	Administración	4	987004321	1999-01-01
Fernando	O	Ordóñez	000004444	1982-09-15	Portillo, s/n	H	36000.00	030445000	5	Investigación	5	033445000	1999-05-22
Aurora	O	Avequela	403453453	1972-07-31	Artón, 5	M	25000.00	030445000	5	Base Central	1	000000000	1901-09-19
Aurora	O	Avequela	403453453	1972-07-31	Artón, 5	M	25000.00	030445000	5	Administración	4	987004321	1999-01-01
Aurora	O	Avequela	403453453	1972-07-31	Artón, 5	M	25000.00	030445000	5	Investigación	5	033445000	1999-05-22
Luis	P	Morera	987957087	1999-09-29	Elnoros, 90	H	25000.00	987954321	4	Base Central	1	000000000	1901-09-19
Luis	P	Morera	987957087	1999-09-29	Elnoros, 90	H	25000.00	987954321	4	Administración	4	987004321	1999-01-01
Luis	P	Morera	987957087	1999-09-29	Elnoros, 90	H	25000.00	987954321	4	Investigación	5	033445000	1999-05-22
Beluanto	O	Paredes	999990000	1997-11-10	Las Pafas, 1	H	00000.00	0	1	Base Central	1	000000000	1901-09-19
Beluanto	O	Paredes	999990000	1997-11-10	Las Pafas, 1	H	00000.00	0	1	Administración	4	987004321	1999-01-01
Beluanto	O	Paredes	999990000	1997-11-10	Las Pafas, 1	H	00000.00	0	1	Investigación	5	033445000	1999-05-22

Observa que da como resultado 24 filas = 8 filas empleado x 3 filas proyecto

Teoría: Unión de tablas

(C) Consideremos el **producto cartesiano** de ambas tablas:

```
SELECT *
FROM empleado, departamento
```

Nombre	Apellido1	Apellido2	Dni	FechaNac	Dirección	Sexo	Sueldo	SuperDni	Dpto	NombreDpto	NumeroDpto	DniDirector	FechaIngresoDirector
José	P	Pérez	123456789	1989-09-01	Eloy I. 90	H	30000.00	030445000	5	Base Central	1	000000000	1901-09-19
José	P	Pérez	123456789	1989-09-01	Eloy I. 90	H	30000.00	030445000	5	Administración	4	987004321	1989-01-01
José	P	Pérez	123456789	1989-09-01	Eloy I. 90	H	30000.00	030445000	5	Investigación	5	033445000	1989-05-22
Alberto	C	Bastie	033445000	1989-12-08	Audix Ríos, 9	H	40000.00	000000000	5	Base Central	1	000000000	1901-09-19
Alberto	C	Bastie	033445000	1989-12-08	Audix Ríos, 9	H	40000.00	000000000	5	Administración	4	987004321	1989-01-01
Alberto	C	Bastie	033445000	1989-12-08	Audix Ríos, 9	H	40000.00	000000000	5	Investigación	5	033445000	1989-05-22
Alicia	J	Delays	999887777	1988-05-12	Gran Vía, 38	M	25000.00	987954321	4	Base Central	1	000000000	1981-09-19
Alicia	J	Delays	999887777	1988-05-12	Gran Vía, 38	M	25000.00	987954321	4	Administración	4	987004321	1989-01-01
Alicia	J	Delays	999887777	1988-05-12	Gran Vía, 38	M	25000.00	987954321	4	Investigación	5	033445000	1989-05-22
Juana	S	Drejs	987654321	1941-05-20	Carquillas, 76	M	43000.00	000000000	4	Base Central	1	000000000	1981-09-19
Juana	S	Drejs	987654321	1941-05-20	Carquillas, 76	M	43000.00	000000000	4	Administración	4	987004321	1989-01-01
Juana	S	Drejs	987654321	1941-05-20	Carquillas, 76	M	43000.00	000000000	4	Investigación	5	033445000	1989-05-22
Fernando	O	Droñez	000004444	1982-09-15	Portillo, s/n	H	36000.00	030445000	5	Base Central	1	000000000	1901-09-19
Fernando	O	Droñez	000004444	1982-09-15	Portillo, s/n	H	36000.00	030445000	5	Administración	4	987004321	1989-01-01
Fernando	O	Droñez	000004444	1982-09-15	Portillo, s/n	H	36000.00	030445000	5	Investigación	5	033445000	1989-05-22
Aurora	O	Avequele	400403403	1972-07-31	Artón, 5	M	25000.00	030445000	5	Base Central	1	000000000	1901-09-19
Aurora	O	Avequele	400403403	1972-07-31	Artón, 5	M	25000.00	030445000	5	Administración	4	987004321	1989-01-01
Aurora	O	Avequele	400403403	1972-07-31	Artón, 5	M	25000.00	030445000	5	Investigación	5	033445000	1989-05-22
Luis	P	Morera	987987087	1969-09-29	Elnoros, 90	H	25000.00	987954321	4	Base Central	1	000000000	1981-09-19
Luis	P	Morera	987987087	1969-09-29	Elnoros, 90	H	25000.00	987954321	4	Administración	4	987004321	1989-01-01
Luis	P	Morera	987987087	1969-09-29	Elnoros, 90	H	25000.00	987954321	4	Investigación	5	033445000	1989-05-22
Beluanto	O	Poncos	888800000	1927-11-10	Las Pafas, 1	H	00000.00	0	1	Base Central	1	000000000	1981-09-19
Beluanto	O	Poncos	888800000	1927-11-10	Las Pafas, 1	H	00000.00	0	1	Administración	4	987004321	1989-01-01
Beluanto	O	Poncos	888800000	1927-11-10	Las Pafas, 1	H	00000.00	0	1	Investigación	5	033445000	1989-05-22

Observa que da como resultado 24 filas = 8 filas empleado x 3 filas proyecto

Teoría: Unión de tablas

Select *

From empleado, departamento

Where empleado.Dno = departamento.NumeroDpto

Antes de Concluir la Sesión

- Subir el modelo de museo
- Subir los scripts de los ejercicios de clase
 - Create
 - Insert

Tarea para el próximo viernes

- Muestra todas las entradas tipo VIP
- Muestra los visitantes que provienen de la ciudad de Xela
- Encuentra las exhibiciones de tipo Escultura que se realizan en la sala Planta baja.
- Muestra los nombres de los visitantes que compraron entradas de tipo VIP.
- Encuentra las exhibiciones de tipo Escultura que se realizan en salas con capacidad mayor a 200 personas.
- Lista los guías que acompañan exhibiciones de tipo Fotografía

Elabora tus queries para obtener estos resultados, utilizando WHERE

Curso: Bases de Datos I
Ingeniería en Sistemas y Ciencias de la
Computación

Clase 11:
Lenguaje SQL: Agrupaciones y filtros en consultas

Catedrático: Claudia Alegre

28 de marzo de 2026

Ejercicio de prácticas...

Red Social de Música Juvenil

Un grupo de jóvenes decide crear una plataforma digital donde puedan compartir sus gustos musicales, organizar eventos y comprar artículos relacionados con sus bandas favoritas. En esta red social, cada usuario se registra con su nombre, edad, ciudad y correo electrónico. Los usuarios pueden crear listas de reproducción, seguir a otros usuarios y comentar en publicaciones musicales. Además, la plataforma organiza eventos juveniles como conciertos, talleres de música y competencias de canto, en los cuales los usuarios pueden inscribirse como asistentes o participantes activos.

La red social también cuenta con una tienda virtual donde los jóvenes pueden adquirir camisetas, gorras, pósters y discos. Cada compra queda registrada con el producto, la categoría, el precio y la cantidad. Los organizadores de eventos son usuarios que tienen permisos especiales para crear y gestionar actividades. Por otro lado, los artistas invitados también se registran en la plataforma y pueden ser vinculados a los eventos en los que participan.

Finalmente, la plataforma permite que los usuarios publiquen reseñas de canciones o álbumes, y que otros usuarios comenten esas reseñas. De esta manera, se genera una comunidad activa que combina interacción social, consumo cultural y participación en actividades juveniles.

*Elabora un modelo de datos entidad relación para este caso en papel y que lo entreguen físico.
30 minutos para hacerlo.*

Dinámica de Introducción (clase anterior)

El Museo Nacional de Arte y Cultura recibe visitantes de distintas ciudades y países. Cada visitante compra una entrada que puede ser General, Estudiante o VIP. El precio varía según el tipo de entrada, y la fecha de compra queda registrada para análisis de ingresos.

El museo organiza exhibiciones en diferentes salas: pintura, escultura, fotografía y piezas de historia natural. Cada exhibición tiene un nombre, un tipo, una sala asignada y un período de duración (fecha de inicio y fecha de fin). Durante su visita, los visitantes recorren una o varias exhibiciones. El museo registra cuánto tiempo pasa cada visitante en cada exhibición, lo que permite analizar la popularidad de las salas y el interés en los distintos tipos de arte.

Además, el museo cuenta con guías que acompañan a los visitantes en recorridos especiales, eventos culturales (conciertos, conferencias, talleres) que se realizan en paralelo a las exhibiciones y una tienda del museo, donde los visitantes pueden comprar recuerdos, libros y catálogos.

Elabora un modelo de datos entidad relación para este caso

Dinámica de Introducción (Clase anterior)

Visitantes

Personas que llegan al museo.

Datos: nombre, apellido, ciudad, país, fecha de visita.

Entradas

Registro de boletos comprados.

Datos: tipo (General, Estudiante, VIP), precio, fecha de compra, visitante asociado.

Exhibiciones

Muestras artísticas o culturales.

Datos: nombre, tipo (Pintura, Escultura, Fotografía, Historia Natural), sala, fechas de inicio y fin.

Visitas_Exhibiciones

Relación entre visitantes y exhibiciones.

Datos: visitante, exhibición, duración de la visita.

Eventos

Actividades culturales adicionales.

Datos: nombre, tipo (Conferencia, Taller, Concierto), fecha, costo.

Tienda

Ventas realizadas en la tienda del museo.

Datos: producto, categoría (Libro, Catálogo, Recuerdo), precio, cantidad, visitante asociado.

Guías

Personal que acompaña a los visitantes en recorridos especiales.

Datos: nombre, idioma, especialidad, horario asignado.

Salas

Espacios físicos donde se ubican las exhibiciones y eventos.

Datos: nombre, capacidad, ubicación (Planta baja, Segundo nivel, Terraza).

Dinámica de Introducción (Clase anterior)

Visitantes

- PK:** visitante_id
- Atributos: nombre, apellido, ciudad, país, fecha_visita

Entradas

- PK:** entrada_id
- FK:** visitante_id → Visitantes
- Atributos: tipo (General, Estudiante, VIP), precio, fecha_compra

Exhibiciones

- PK:** exhibicion_id
- FK:** sala_id → Salas
- Atributos: nombre, tipo (Pintura, Escultura, Fotografía, Historia Natural), fecha_inicio, fecha_fin

Visitas_Exhibiciones

- PK:** visita_id
- FK:** visitante_id → Visitantes
- FK:** exhibicion_id → Exhibiciones
- Atributos: duración_minutos

Eventos

- PK:** evento_id
- FK:** sala_id → Salas
- Atributos: nombre, tipo (Conferencia, Taller, Concierto), fecha, costo

Tienda

- PK:** venta_id
- FK:** visitante_id → Visitantes
- Atributos: producto, categoría (Libro, Catálogo, Recuerdo), precio, cantidad

Guías

- PK:** guia_id
- Atributos: nombre, idioma, especialidad, horario_asignado

Salas

- PK:** sala_id
- Atributos: nombre, capacidad, ubicación (Planta baja, Segundo nivel, Terraza)

Tema

Lenguaje SQL: Agrupaciones y filtros en consultas

Indicadores de Logro

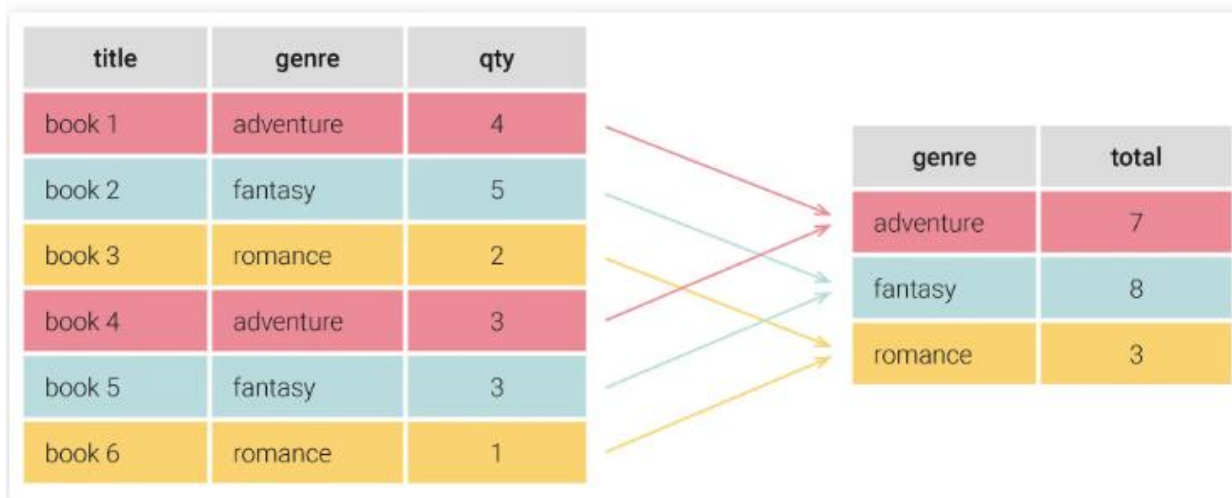
- Los estudiantes serán capaces de utilizar la cláusula *GROUP BY* en consultas SQL para agrupar datos y calcular funciones de agregación sobre ellos.
- Los estudiantes serán capaces de utilizar funciones de agregación (como *SUM*, *COUNT*, *AVG*, etc), manipulación de fechas (como *DATE_ADD*, *DATE_SUB*, etc) u operadores lógicos (como *AND*, *OR*, *NOT*) y aritméticos (como +, -, *, /) en consultas SQL para manipular y comparar datos.
- Los estudiantes serán capaces de combinar funciones y operadores

Actividades

1. Explicación de la teoría
2. Ejemplos
3. Ejercicios prácticos

Teoría: Group by

¿Qué hace GROUP BY? **agrupa las filas con el mismo valor en un grupo**



```
SELECT genre, SUM(qty) AS total  
FROM books  
GROUP BY genre;
```

Practiquemos

- Calcula el ingreso total del museo agrupado por tipo de entrada (*General, Estudiante, VIP*)
- Cuenta cuántos visitantes llegaron al museo agrupados por ciudad de origen.
- Lista el número de exhibiciones y eventos que se realizan en cada sala

Elabora tus queries para obtener estos resultados, utilizando group by

Teoría: Group by

SUM, COUNT, AVG

SELECT

Categoria,

COUNT(*) AS TotalProductos,

SUM(Stock) AS InventarioTotal,

MAX(Precio) AS PrecioMasAlto,

AVG(Precio) AS PrecioPromedio

FROM Productos

GROUP BY Categoria;

Practiquemos

Resuelve las siguientes necesidades:

- Calcula el número total de visitantes registrados.
- Obtén el ingreso total de la tienda ($\text{precio} \times \text{cantidad}$).
- Promedio de duración de las visitas a exhibiciones.

Teoría: Fechas

Manipulación de fechas : DATE_ADD, DATE_SUB

```
SELECT DATEADD(day, 10, GETDATE()) AS Sumar_10_Días;  
SELECT DATEADD(month, -2, GETDATE()) AS Restar_2_Meses;  
SELECT DATEADD(year, 5, '2020-01-01') AS Fecha_Futura;
```

```
SELECT date_sub(YEAR, 1, to_date('2018-01-02'));
```

2017-01-02

Practiquemos

Resuelve las siguientes necesidades:

- Lista las exhibiciones cuya **fecha de fin** se extienda **30 días después** de su fecha de inicio.
- Lista las salas que tendrán eventos programados **90 días después de hoy**.

Teoría: Operadores

Operadores Lógicos: AND, OR, NOT

```
SELECT first_name, last_name  
FROM Customers  
WHERE country = 'USA' AND last_name = 'Doe';
```

```
SELECT first_name, last_name  
FROM Customers  
WHERE country = 'USA' OR last_name = 'Doe';
```

```
SELECT first_name, last_name  
FROM Customers  
WHERE NOT country = 'USA';
```

Teoría: Operadores

Operadores aritméticos +, -, *, /

```
SELECT 60 * 60 * 24 * 7;  
-- resultado: 604800
```

```
SELECT 25 / 4;  
-- resultado: 6
```

Practiquemos

Resuelve las siguientes necesidades:

- Lista los visitantes que son de Guatemala Y cuya fecha de visita fue en 2026.
- Encuentra los visitantes que son de México O El Salvador.
- Muestra los visitantes que NO compraron entradas de tipo “Estudiante”.
- Obtén las entradas de tipo “VIP” Y con precio mayor a 100.
- Muestra el precio de las entradas con un **descuento del 10%** ($\text{precio} - (\text{precio} * 0.10)$).
- Convierte la duración de las visitas en **horas** ($\text{duración_minutos} / 60$).

Antes de Concluir la Sesión

- Subir los scripts de los ejercicios de clase

Tarea para el Viernes 10 abril 2026

La **Biblioteca de la Universidad** administra préstamos de libros, revistas y material digital. Los estudiantes y profesores son usuarios frecuentes, y cada uno debe registrarse para poder realizar préstamos. La biblioteca también organiza actividades culturales como charlas y clubes de lectura.

Entregar:

Modelo de datos en gráfico

Create tables

Insert (poblar)

Scripts utilizando where y group by respondiendo lo siguiente:

- Muestra todos los préstamos que están en estado *Retrasado*.
- Lista los libros del género *Historia* publicados después del año 2015.
- Lista la cantidad de libros que prestan los estudiantes por nombre y apellido
- Obtén el número de libros agrupados por género literario

Curso: Bases de Datos I

Ingeniería en Sistemas y Ciencias de la Computación

Clase 13:

Diseño de Base de datos y consultas

Catedrático: Claudia Alegre

11 de abril de 2026

Tema

Ejercicios prácticos: Diseño de bases de datos

Indicadores de Logro

- Los estudiantes serán capaces de diseñar una base de datos relacional eficiente y normalizada utilizando herramientas y técnicas aprendidas en clase.

Actividades

1. Explicación de la teoría
2. Ejemplos
3. Ejercicios prácticos

Practiquemos....

Caso: Elecciones Generales en Guatemala

En Guatemala se realizan elecciones generales cada cuatro años para elegir Presidente y Vicepresidente de la República, Diputados al Congreso y Alcaldes municipales. El Tribunal Supremo Electoral (TSE) necesita un **sistema de bases de datos** que permita gestionar todo el proceso de manera segura, transparente y auditable.

- **Registro de ciudadanos habilitados para votar:** Cada ciudadano debe estar inscrito con un identificador único, datos personales básicos (nombre, DPI, municipio de residencia), y estado de habilitación para votar. Restricción: un ciudadano solo puede emitir un voto por cada tipo de elección (Presidente, Diputados, Alcalde).
- **Candidatos y cargos:** Los partidos políticos inscriben candidatos para los distintos cargos. Se debe registrar: nombre del candidato, partido político, cargo al que aspira (Presidente, Diputado, Alcalde), distrito o municipio correspondiente, y fecha de inscripción.
- **Proceso de votación:** Cada ciudadano puede emitir votos electrónicos o físicos que luego se registran en el sistema. El sistema debe garantizar que no existan votos duplicados y que cada voto quede asociado al cargo y al distrito/municipio correspondiente.
- **Resultados y auditoría:** El sistema debe permitir calcular el total de votos por candidato y cargo, tanto a nivel nacional como por distrito y municipio. Se deben registrar eventos relevantes (intentos de fraude, votos anulados, inconsistencias en actas) para fines de auditoría.
- **Seguridad y control:** Se requiere que el sistema tenga mecanismos para validar la integridad de los datos, restringir accesos según roles (administrador, auditor, digitador), y mantener trazabilidad de todas las operaciones.

Actividades a realizar:

- **Identificar entidades principales**

Solución entidades

1. Ciudadanos habilitados para votar
2. Partidos políticos
3. Candidatos
4. Cargos en elección
5. Votos emitidos
6. Auditoría y control
7. Distritos electorales
8. Mesas electorales / Centros de votación
9. Elecciones (evento electoral)
10. Resultados oficiales
11. Actas electorales
12. Usuarios del sistema

Actividades a realizar:

- **Definamos atributos**

Solución

- **Ciudadanos habilitados para votar**

Identificador único (DPI)
Nombre completo
Fecha de nacimiento
Municipio/Distrito de residencia
Estado de habilitación (activo, suspendido, no habilitado)

- **Partidos políticos**

Identificador del partido
Nombre del partido
Siglas
Ideología/Descripción
Fecha de inscripción en el TSE
Estado (vigente, cancelado)

- **Candidatos**

Identificador del candidato
Nombre completo
Cargo al que aspira (Presidente, Diputado, Alcalde)
Partido político al que pertenece
Distrito/Municipio correspondiente
Fecha de inscripción
Plan de trabajo (resumen)

- **Cargos en elección**

Identificador del cargo
Tipo de cargo (Presidente, Diputado, Alcalde)
Nivel (nacional, distrital, municipal)
Período de elección (ej. 2027–2031)

- **Votos emitidos**

Identificador del voto
Identificador del ciudadano
Identificador del candidato
Cargo asociado
Fecha y hora del voto
Estado del voto (válido, nulo, duplicado)

- **Auditoría y control**

Identificador del evento
Tipo de evento (acceso, intento de fraude, voto anulado, inconsistencia en acta)
Usuario o ciudadano relacionado
Fecha y hora del evento
Observaciones/Detalle

- **Distritos electorales**

Identificador del distrito
Nombre del distrito
Departamento/Región
Número de municipios asociados

- **Mesas electorales / Centros de votación**

Identificador de la mesa
Ubicación (municipio, dirección)
Distrito asociado
Número de ciudadanos asignados

- **Elecciones (evento electoral)**

Identificador de la elección
Tipo de elección (general, municipal, legislativa)
Fecha de inicio y cierre
Estado (programada, en curso, finalizada)

- **Resultados oficiales**

Identificador del resultado
Cargo
Candidato
Total de votos obtenidos
Distrito/Municipio
Estado de validación (preliminar, oficial)

- **Actas electorales**

Identificador del acta
Mesa electoral asociada
Número de votos registrados
Número de votos válidos, nulos y en blanco
Firma de responsables

- **Usuarios del sistema**

Identificador de usuario
Nombre completo
Rol (administrador, auditor, digitador, observador)
Credenciales de acceso
Estado (activo, inactivo)

Actividades a realizar:

- **Definir relaciones** entre ellas (por ejemplo, un ciudadano emite un voto para un candidato en un cargo; un partido político inscribe varios candidatos).

Solución

- **Ciudadano** Cada ciudadano está inscrito en el padrón electoral y **puede emitir votos**. Relación: Un ciudadano emite uno o varios votos, pero solo uno por cada cargo.
- **Partido político** Los partidos son organizaciones que **inscriben candidatos**. Relación: Un partido político inscribe muchos candidatos; cada candidato pertenece a un partido.
- **Candidato** Es la persona que compete por un cargo. Relación: Un candidato se postula a un cargo específico; un cargo puede tener varios candidatos.
- **Cargo en elección** Define la posición a elegir (Presidente, Diputado, Alcalde). Relación: Un cargo recibe votos de ciudadanos a través de los candidatos inscritos.
- **Voto emitido** Es la acción de un ciudadano al elegir un candidato. Relación: Cada voto vincula a un ciudadano con un candidato en un cargo, en una elección. Cada voto vincula a un ciudadano con un candidato en un cargo, en una elección específica.
- **Elección (evento electoral)** Agrupa los cargos, candidatos y votos de un proceso electoral. Relación: Una elección contiene varios cargos; cada cargo tiene candidatos; cada candidato recibe votos.
- **Distrito electoral / Municipio** Representa la división geográfica. Relación: Un ciudadano pertenece a un distrito; un candidato puede postularse en un distrito; los votos se contabilizan por distrito.
- **Mesa electoral / Centro de votación** Lugar físico donde se emiten los votos. Relación: Una mesa recibe votos de ciudadanos asignados; cada mesa pertenece a un distrito.
- **Acta electoral** Documento que registra los resultados de una mesa. Relación: Un acta resume los votos emitidos en una mesa; se vincula con los resultados oficiales.
- **Resultados oficiales** Consolidación de votos por cargo y candidato. Relación: Los resultados se derivan de las actas y se validan en la elección.
- **Auditoría y control** Registra accesos, intentos de fraude y votos anulados. Relación: La auditoría se vincula con ciudadanos, votos y candidatos para garantizar transparencia.
- **Usuarios del sistema** Roles como administrador, auditor, digitador. Relación: Un usuario interactúa con el sistema y genera registros en auditoría.

Actividades a realizar:

- **Diseño ER**

Actividades a realizar:

Diseñar consultas que respondan preguntas clave:

- ¿Cuántos votos obtuvo cada candidato a Presidente?
- ¿Qué partido político obtuvo más diputados por distrito?
- ¿Qué municipio tuvo mayor participación ciudadana?
- ¿Existen votos duplicados o inconsistencias en actas?

Actividades a realizar:

- **Proponer roles de acceso** en la base de datos
(administrador, auditor, digitador)

Antes de Concluir la Sesión

- Subir los scripts de los ejercicios de clase
- Subir tu modelo relacional

Tarea para el Viernes 17 abril 2026

El Ministerio de Educación de Guatemala lanza el programa **Avanza Guatemala**, que otorga becas a estudiantes de nivel medio y universitario en todo el país. El sistema debe permitir registrar solicitudes, asignar becas, controlar pagos y evaluar el desempeño académico de los beneficiarios.

Registro de estudiantes: Cada estudiante debe estar inscrito con un número de identificación único, datos personales, nivel educativo, institución y municipio de residencia.

Convocatorias de becas: Cada año se abren varias convocatorias con requisitos específicos (nivel, área de estudio, promedio mínimo, fecha de apertura y cierre).

Solicitudes: Los estudiantes pueden aplicar a una o más convocatorias, pero solo pueden recibir una beca activa por período. Cada solicitud incluye fecha, estado (pendiente, aprobada, rechazada) y documentos adjuntos.

Asignación de becas: Una vez aprobada la solicitud, se genera un registro de beca con monto, duración, tipo (académica, deportiva, artística) y estado (activa, suspendida, finalizada).

Pagos: Las becas se pagan mensualmente. Cada pago debe registrar fecha, monto, medio de transferencia y estado (realizado, pendiente, retenido).

Evaluaciones académicas: Los estudiantes beneficiarios deben presentar evaluaciones periódicas con promedio y observaciones del tutor o institución.

Instituciones educativas: Cada institución tiene código, nombre, tipo (colegio, universidad), municipio y contacto.

Entregar:

Modelo de datos en gráfico

Curso: Bases de Datos I

Ingeniería en Sistemas y Ciencias de la Computación

Clase 16:

Triggers y Package procedure

Catedrático: Claudia Alegre

2 de mayo de 2026

Dinámica introducción



3 equipos:

- Equipo datos: escribe nombres, actualiza nombres, elimina nombres
- Equipo eventos: Indica acción Insertar, Actualizar, eliminar
- Equipo Trigger: escribe bitácora
 - Insertó nombre, hora min
 - Actualizó nombre, hora min
 - Eliminó nombre, hora min



Tema

Triggers y Package Procedure

Indicadores de Logro

- Los estudiantes serán capaces de utilizar triggers y Procedimientos empacados en SQL para resolver problemas complejos de consulta.
- Los estudiantes serán capaces de evaluar la eficiencia utilizando disparadores y package dentro de la base de datos

Actividades

1. Explicación de la teoría
2. Ejemplos
3. Ejercicios prácticos

Teoría – ¿Qué es un trigger

- Un desencadenador es un tipo especial de procedimiento almacenado que se ejecuta automáticamente cuando se produce un evento en el servidor de bases de datos.
- CREATE TRIGGER NombreTrigger
- ON NombreTabla
- AFTER INSERT, UPDATE, DELETE -- O INSTEAD OF AS
- BEGIN -- Lógica del trigger
- END;

Desencadenadores DML

El desencadenador DML es un tipo especial de procedimiento almacenado que surte efecto automáticamente cuando se produce un evento de lenguaje de manipulación de datos (DML) que afecta a la tabla o vista definida en el desencadenador. Los eventos DML incluyen
las instrucciones INSERT,
las instrucciones UPDATE, o
las instrucciones DELETE

El desencadenador y la instrucción que lo activa se tratan como una sola transacción, que puede revertirse desde el desencadenador.

Ejemplo

Crear tabla alumnos y auditoría

Alumno

```
Carnet nchar(10)
nombre nvarchar(50)
apellido nvarchar(50)
```

Auditoria

```
fecha date
nombre nvarchar(50)
```

Crear trigger

```
Create trigger Control_alu
on alumno
After insert as
Begin
    Insert into auditoria
        Select getdate(), i.nombre
        from inserted i
end
```

Tipos de eventos

Un **Trigger** se ejecuta automáticamente cuando ocurre un evento específico en una tabla. Estos eventos pueden ser INSERT, DELETE y UPDATE.

Dentro de un trigger, SQL Server crea dos tablas temporales en memoria que contienen los datos afectados por la operación:

1.inserted: Contiene las filas nuevas (en un INSERT) o las versiones nuevas de las filas (en un UPDATE).

2.deleted: Contiene las filas eliminadas (en un DELETE) o las versiones antiguas de las filas (en un UPDATE).

OPERACIÓN	TABLA INSERTED	TABLA DELETED
INSERT	Las nuevas filas	(Vacía)
DELETE	(Vacía)	Las filas borradas
UPDATE	Las filas con los nuevos valores	Las filas con los viejos valores

Ejercicio

Registro Automático de Cambios

Tienes una tabla de usuarios. Cada vez que se inserte un nuevo usuario, queremos que automáticamente se guarde un registro en una tabla de auditoría indicando la acción realizada.

1. Crear las tablas

-- Tabla principal

```
CREATE TABLE Usuarios (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  nombre VARCHAR(50),  
  edad INT  
);
```

-- Tabla de auditoría

```
CREATE TABLE Auditoria (  
  id INT PRIMARY KEY AUTO_INCREMENT,  
  accion VARCHAR(50),  
  fecha TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

2. Crear el Trigger

```
CREATE TRIGGER after_insert_usuario  
AFTER INSERT ON Usuarios  
FOR EACH ROW  
BEGIN  
  INSERT INTO Auditoria (accion)  
  VALUES (CONCAT('Se insertó el usuario: ', NEW.nombre));  
END;
```

3. Probar el Trigger

-- Insertamos un nuevo usuario

```
INSERT INTO Usuarios (nombre, edad) VALUES ('Ana', 22);
```

-- Revisamos la tabla de auditoría

```
SELECT * FROM Auditoria;
```


Ejercicio

Guardar Usuarios Eliminados

Queremos que cada vez que se elimine un usuario de la tabla Usuarios, automáticamente se guarde su información en una tabla llamada UsuariosEliminados. Así no se pierde el historial.

Solución

Crear la tabla de respaldo

```
CREATE TABLE UsuariosEliminados (  
  id INT,  
  nombre VARCHAR(50),  
  edad INT,  
  fecha_eliminacion TIMESTAMP DEFAULT CURRENT_TIMESTAMP  
);
```

2. Crear el Trigger

```
CREATE TRIGGER after_delete_usuario  
AFTER DELETE ON Usuarios  
FOR EACH ROW  
BEGIN  
  INSERT INTO UsuariosEliminados (id, nombre, edad)  
  VALUES (OLD.id, OLD.nombre, OLD.edad);  
END;
```

3. Probar el Trigger

```
-- Insertamos un usuario  
INSERT INTO Usuarios (nombre, edad) VALUES ('Lucía', 28);  
  
-- Eliminamos el usuario  
DELETE FROM Usuarios WHERE nombre = 'Lucía';  
  
-- Revisamos la tabla de respaldo  
SELECT * FROM UsuariosEliminados;
```

Caso

Actualizar Stock Automáticamente

Tienes una tabla de productos con su cantidad en inventario. Cada vez que se inserte un pedido en la tabla Pedidos, queremos que el trigger automáticamente reste la cantidad pedida del stock del producto correspondiente.

Solución

1. Crear las tablas

-- Tabla de productos

```
CREATE TABLE Productos (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    nombre VARCHAR(50),  
    stock INT  
);
```

-- Tabla de pedidos

```
CREATE TABLE Pedidos (  
    id INT PRIMARY KEY AUTO_INCREMENT,  
    producto_id INT,  
    cantidad INT,  
    FOREIGN KEY (producto_id) REFERENCES  
    Productos(id)  
);
```

2. Crear el Trigger

```
CREATE TRIGGER after_insert_pedido  
AFTER INSERT ON Pedidos  
FOR EACH ROW  
BEGIN  
    UPDATE Productos  
    SET stock = stock - NEW.cantidad  
    WHERE id = NEW.producto_id;  
END;
```

3. Probar el Trigger

-- Insertamos un producto

```
INSERT INTO Productos (nombre, stock) VALUES  
('Laptop', 10);
```

-- Insertamos un pedido

```
INSERT INTO Pedidos (producto_id, cantidad) VALUES  
(1, 3);
```

-- Revisamos el stock

```
SELECT * FROM Productos;
```

Procedimientos almacenados

Los **procedimientos almacenados** son conjuntos de una o varias instrucciones almacenadas en base de datos, que pueden ser llamados desde otra parte de la aplicación. Los **procedimientos almacenados** permiten recibir parámetros de entrada y/o salida, aunque no siempre es necesario que devuelvan información. Además, y a diferencia de las funciones, pueden realizar operaciones internas que alteren la base de datos tales como INSERTS, UPDATES y/o DELETES.

Características y tipos

- Permiten recibir parámetros de entrada y/o salida.
- Pueden realizar operaciones internas que alteren el estado de la base de datos (**INSERT INTO...**)
- Pueden llamar a otros **procedimientos almacenados**.
- Al encapsular bloques de código, mejoran el tráfico de red entre cliente y servidor. Además, mejora la eficiencia al estimular la reutilización de código.
- Aportan mayor seguridad. Por ejemplo, los procedimientos almacenados pueden cifrarse y el uso de parámetros protege contra inyecciones directas de código **SQL**.
- Mejoran el rendimiento y facilitan el mantenimiento

Antes de Concluir la Sesión

- Subir los scripts de los ejercicios de clase
- Qué aprendimos hoy?

Tarea para el Viernes 8 mayo 2026

Consulta de Producto por Id

Tabla

Productos

IdProducto (PK)

Nombre

Precio

Requerimiento

Crear un **procedimiento almacenado** que permita **consultar la información de un producto** a partir de su Id.

Reglas:

Recibir como parámetro el Id del producto.

Si el producto existe, mostrar su nombre y precio.

Si no existe, mostrar un mensaje indicando que no se encontró.

ENTREGABLES:

- Crear la tabla Productos con algunos registros de prueba.
- Implementar el procedimiento ConsultarProductoPorId.
- Probarlo con distintos casos:
 - Producto existente (debe mostrar datos).
 - Producto inexistente (debe mostrar mensaje de error).

Curso: Bases de Datos I

Ingeniería en Sistemas y Ciencias de la Computación

Clase 17: **Transact SQL**

Catedrático: Claudia Alegre

9 de mayo de 2026

Dinámica introducción



Comprendiendo T-SQL

SQL es el lenguaje estándar para consultar y manipular bases de datos relacionales.

T-SQL es la versión extendida de SQL que usa Microsoft SQL Server.

T-SQL incluye todo lo que hace SQL, pero además agrega **estructuras de programación** como variables, condiciones, ciclos, procedimientos almacenados y manejo de errores.

Definición:

Explica con tus propias palabras qué es SQL.

Explica qué es T-SQL y por qué se considera una extensión.

Solución

Aspecto	SQL estándar	T-SQL (SQL Server)
Uso principal	Consultar y manipular datos	Consultar + programar lógica
Variables	No soporta	Sí, con <code>DECLARE</code> y <code>SET</code>
Control de flujo	No soporta	Sí, con <code>IF</code> , <code>WHILE</code> , <code>BEGIN...END</code>



Tema

Transact SQL

Indicadores de Logro

- Los estudiantes serán capaces de utilizar variables en consultas SQL para mejorar la flexibilidad y reutilización de código.
- Los estudiantes serán capaces de evaluar la conveniencia de utilizar variables en lugar de valores fijos.

Actividades

1. Explicación de la teoría
2. Ejemplos
3. Ejercicios prácticos

Teoría – Transact SQL

- Transact-SQL (T-SQL) proporciona un lenguaje de programación sólido con características que permiten almacenar temporalmente valores en variables, aplicar la ejecución condicional de comandos, pasar parámetros a procedimientos almacenados y controlar el flujo de los programas.

Transact SQL

Transact-SQL (T-SQL) es una extensión propietaria del lenguaje de consulta estructurado (SQL) estándar abierto. Admite variables declaradas, procesamiento de cadenas y datos, control de errores y excepciones y control de transacciones. Aunque SQL es un lenguaje de programación, T-SQL agrega compatibilidad con la programación de procedimientos y el uso de variables locales.

Normalmente, un programa de T-SQL comenzará con una instrucción BEGIN y finalizará con una instrucción END, con las instrucciones que desee ejecutar entre ellas

Transact SQL

Estas son algunas de las estructuras de programación de T-SQL admitidas:

- ✓ **IF.. ELSE:** instrucción condicional que permite decidir qué aspectos del código se ejecutarán.
- ✓ **WHILE:** instrucción de bucle ideal para ejecutar iteraciones de instrucciones T-SQL.
- ✓ **DECLARE** - Vas a usarlo para definir variables.
- ✓ **SET:** una de las formas en que asignará valores a las variables.
- ✓ **BATCHES:** serie de instrucciones T-SQL que se ejecutan como una unidad.

Variables

En T-SQL, al igual que con otros lenguajes de programación, las variables son objetos que permiten el almacenamiento temporal de un valor para su uso posterior. Ya ha visto variables cuando las ha usado para pasar valores de parámetro a procedimientos almacenados y funciones.

En T-SQL, las variables deben declararse para poder usarlas. Se les puede asignar un valor o inicializarse cuando se declaran. Declarar una variable incluye proporcionar un nombre y un tipo de datos, como se muestra a continuación. Para declarar una variable, debe usar la instrucción DECLARE

Variables

Una vez que haya declarado una variable, debe inicializarla o asignarle un valor. Puede hacerlo de tres maneras:

- En SQL Server 2008 o posterior, puede inicializar una variable mediante la instrucción DECLARE.
- En cualquier versión de SQL Server, puede asignar un valor único (escalar) mediante la instrucción SET.
- En cualquier versión de SQL Server, puede asignar un valor a una variable mediante una instrucción SELECT. Asegúrese de que la instrucción SELECT devuelve exactamente una fila. Un resultado vacío dejará la variable con su valor original; más de un resultado devolverá el último resultado.

En el ejemplo siguiente se muestran las tres formas de declarar y asignar valores a variables:

```
DECLARE @var1 AS INT = 99;  
DECLARE @var2 AS NVARCHAR(25);  
SET @var2 = N'Ejemplo';  
DECLARE @var3 AS NVARCHAR(20);  
SELECT @var3 = apellido FROM alumno WHERE carnet=1;  
SELECT @var1 AS variable1, @var2 AS variable2, @var3 AS variable3;  
GO
```

Ejercicios

tabla Clientes

```
CREATE TABLE Clientes (  
    ClienteID INT PRIMARY KEY,  
    Nombre NVARCHAR(50),  
    Ciudad NVARCHAR(50),  
    Edad INT  
);
```

Poblemos la tabla

1. Selecciona todos los registros
2. Selecciona solo los nombres y ciudades
3. Filtra clientes de Guatemala
4. Filtra clientes mayores de 30 años
5. Ordenar por edad descendente
6. Contar cuantos clientes hay en total
7. Agrupar por ciudad y contar clientes de cada una de las ciudades

Ejercicio

tabla de productos llamada Inventario:

sql

```
CREATE TABLE Inventario (  
    ProductoID INT PRIMARY KEY,  
    Nombre NVARCHAR(50),  
    Stock INT  
);
```

```
INSERT INTO Inventario (ProductoID, Nombre, Stock)  
VALUES  
(1, 'Laptop', 5),  
(2, 'Mouse', 20),  
(3, 'Teclado', 15);  
.....
```

Problema

Queremos simular la **venta de unidades de un producto**.

- Si el stock es suficiente, se descuenta.
- Si el stock llega a cero, se detiene la venta y se muestra un mensaje.
- Usaremos un ciclo `WHILE` para simular varias ventas consecutivas.

Solución

```
DECLARE @ProductoID INT = 1;           -- Producto: Laptop
DECLARE @StockActual INT;
DECLARE @Venta INT = 1;                 -- Cada venta descuenta 1 unidad

-- Obtener stock inicial
SELECT @StockActual = Stock
FROM Inventario
WHERE ProductoID = @ProductoID;

-- Simular ventas hasta que se acabe el stock
WHILE @StockActual > 0
BEGIN
    SET @StockActual = @StockActual - @Venta;

    IF @StockActual >= 0
        PRINT 'Venta realizada. Stock restante: ' + CAST(@StockActual AS
NVARCHAR(10));
    ELSE
        PRINT 'No hay suficiente stock para continuar.';
END

-- Actualizar la tabla con el stock final
UPDATE Inventario
SET Stock = @StockActual
WHERE ProductoID = @ProductoID;
```

Ejercicio 2

Validación de pedidos de clientes

Supongamos que tenemos estas tablas:

sql

```
CREATE TABLE Clientes (  
    ClienteID INT PRIMARY KEY,  
    Nombre NVARCHAR(50),  
    Estado NVARCHAR(20)    -- Activo o Inactivo  
);  
  
CREATE TABLE Pedidos (  
    PedidoID INT PRIMARY KEY,  
    ClienteID INT,  
    FechaPedido DATE,  
    Monto DECIMAL(10,2),  
    FOREIGN KEY (ClienteID) REFERENCES Clientes(ClienteID)  
);
```

Problema

Queremos procesar pedidos de un cliente específico:

- Si el cliente está **Inactivo**, no se procesan pedidos.
- Si está **Activo**, se recorren sus pedidos con un ciclo `WHILE`.
- Se acumula el total gastado y se imprime cada pedido.
- Al final se muestra el total acumulado.

Solución

```
DECLARE @ClienteID INT = 1;          -- Cliente a validar
DECLARE @Estado NVARCHAR(20);
DECLARE @TotalGastado DECIMAL(10,2) = 0;
DECLARE @PedidoID INT;
DECLARE @Monto DECIMAL(10,2);

-- Obtener estado del cliente
SELECT @Estado = Estado
FROM Clientes
WHERE ClienteID = @ClienteID;

IF @Estado = 'Inactivo'
BEGIN
    PRINT 'El cliente está inactivo. No se procesan pedidos.';
END
ELSE
BEGIN
    PRINT 'Procesando pedidos del cliente...';

    -- Cursor para recorrer pedidos
    DECLARE PedidoCursor CURSOR FOR
        SELECT PedidoID, Monto
        FROM Pedidos
        WHERE ClienteID = @ClienteID;

    OPEN PedidoCursor;
    FETCH NEXT FROM PedidoCursor INTO @PedidoID, @Monto;

    WHILE @@FETCH_STATUS = 0
    BEGIN
        PRINT 'Pedido ' + CAST(@PedidoID AS NVARCHAR(10)) +
            ' procesado. Monto: ' + CAST(@Monto AS NVARCHAR(10));

        SET @TotalGastado = @TotalGastado + @Monto;

        FETCH NEXT FROM PedidoCursor INTO @PedidoID, @Monto;
    END

    CLOSE PedidoCursor;
    DEALLOCATE PedidoCursor;

    PRINT 'Total gastado por el cliente: ' + CAST(@TotalGastado AS NVARCHAR(10));
END
```


Antes de Concluir la Sesión

- Subir los scripts de los ejercicios de clase
- Qué aprendimos hoy?

Tarea para el Viernes 15 mayo 2026

Control de crédito de clientes

La empresa tiene una tabla de clientes con un **límite de crédito**.

Cada vez que se registra un pedido, se descuenta del crédito disponible.

Si el crédito llega a cero, el cliente ya no puede hacer más pedidos.

Problema a resolver

1. Declara una variable para el **ClienteID** y otra para el **monto de cada pedido**.
2. Usa un `IF` para verificar si el cliente tiene crédito disponible antes de procesar pedidos.
3. Si tiene crédito, usa un `WHILE` para simular varios pedidos consecutivos (por ejemplo, cada pedido cuesta 120).
4. En cada iteración, descuenta el monto del crédito y muestra el crédito restante.
5. Cuando el crédito ya no alcance para cubrir el pedido, el ciclo debe detenerse y mostrar un mensaje indicando que el cliente no puede seguir comprando.